

SIH SMART AUTOMATION - COMPLETE TECH STACK ANALYSIS

Comprehensive technology stack recommendations for all Smart Automation theme problem statements across SIH 2023, 2024, and 2025/2026. Each problem includes a standard competition-winning tech stack and a separate advanced/buzzword section.

SIH 2025 / 2026 (12 Statements)

PS No.	Organization	Cat.	Problem Statement	Standard Tech Stack (Competition-Winning)	Advanced / Buzzword Tech
SIH25020	Ministry of Railways	Hardware	Development of indigenous contactless Integrated Track Monitoring Systems (ITMS) for Track Recording on Indian Railways	Hardware: STM32/ESP32 MCU, IMU sensors (MPU6050/ADXL345), LVDT displacement sensors, laser profilometers, ultrasonic flaw detectors (UT probes), vibration/accelerometer sensors, GPS module (NEO-6M/u-blox), LoRa/4G communication module, LiPo battery + solar charging circuit, custom PCB (KiCad/Altium), ruggedized IP67 enclosure Edge Computing: Raspberry Pi 4/5 or NVIDIA Jetson Nano for on-device ML inference Firmware: C/C++ (FreeRTOS or bare-metal), Arduino framework for prototyping Signal Processing: Python (NumPy, SciPy) for offline calibration; DSP libraries on MCU ML/AI: TensorFlow Lite / TinyML for anomaly detection on edge; scikit-learn for model training Backend: Python (FastAPI/Flask), PostgreSQL + TimescaleDB for time-series data Dashboard: React.js, Chart.js/D3.js, Leaflet.js for geo-mapping track defects Communication: MQTT protocol, LoRaWAN gateway, GSM/4G fallback DevOps: Docker, GitHub Actions CI/CD	AIoT Edge Intelligence, Predictive Maintenance 4.0, Digital Twin Track Simulation, Federated Learning for distributed sensor nodes, Blockchain-based tamper-proof maintenance logs, 5G-NR URLLC for ultra-reliable low-latency track alerts
SIH25033	Ministry of Corporate Affairs	Software	AI-Based Smart Allocation Engine for PM Internship Scheme	Frontend: React.js / Next.js, HTML5, CSS3, JavaScript (ES6+), Axios Backend: Python (Django REST Framework / FastAPI), Celery for async task processing Database: PostgreSQL (primary), Redis (caching + job queue) AI/ML: scikit-learn (classification, clustering), XGBoost / LightGBM for ranking, pandas, NumPy for data processing, NLTK/spaCy for resume parsing Matching Algorithm: Hungarian Algorithm / Gale-Shapley for optimal allocation, cosine similarity + TF-IDF for skill matching Authentication: JWT, OAuth 2.0, DigiLocker API integration Deployment: Docker, Nginx, Gunicorn, AWS EC2/ECS or NIC Cloud Testing: pytest, Selenium for E2E tests	Transformer-based Skill Embeddings (BERT/IndicBERT), Graph Neural Networks for applicant-internship matching, Reinforcement Learning for adaptive allocation optimization, Explainable AI (SHAP/LIME) for transparent decision-making
SIH25043	Ministry of Earth Sciences (MoES)	Hardware	Embedded Intelligent Microscopy System for Identification and Counting of Microscopic Marine Organisms	Hardware: Raspberry Pi 4/5, Pi Camera Module v3 (HQ) or USB microscope camera, motorized XY stage (stepper motors + A4988 drivers), LED ring illumination, custom 3D-printed microscope mount, Arduino Nano for stage control Optics: Brightfield/darkfield illumination, 10x-40x objective lenses, achromatic lens assembly Computer Vision: OpenCV, YOLOv8/YOLOv9 for object detection, U-Net for segmentation of organisms ML Framework: PyTorch / TensorFlow, ONNX Runtime for edge inference, dataset: annotated marine plankton images (WHOI-Plankton dataset as baseline) Backend: Python (Flask), SQLite for local storage, JSON export GUI: PyQt5 / Tkinter for on-device interface, or React.js web dashboard Communication: Wi-Fi (built-in RPi), optional 4G HAT for remote areas	Vision Transformer (ViT) for plankton classification, Few-Shot Learning for rare species, Edge TPU acceleration (Google Coral), Automated Microscopy Digital Twin, Citizen Science API integration for crowd-validated taxonomy
SIH25054	Ministry of Consumer Affairs, Food & Public Distribution	Hardware	Automated High-Current Short-Circuit Test System for MCB to comply with IEC 60898-1:2015	Hardware: High-current transformer (adjustable up to 10kA), programmable power supply, current shunt resistors + Hall-effect sensors (ACS758), voltage dividers, high-speed DAQ (NI DAQ / ADS1256 24-bit ADC), relay switching matrix, solenoid-actuated test fixture, safety interlock system Microcontroller: STM32F4 / Arduino Mega for test sequencing and relay control Instrumentation: Oscilloscope integration (SCPI protocol), Rogowski coils for transient current Software: Python (PyVISA for instrument control, NumPy/SciPy for waveform analysis), LabVIEW alternative: Python + NI-DAQmx GUI: PyQt5 / Electron.js for test control panel Data & Reports: SQLite/PostgreSQL, ReportLab/Matplotlib for auto-generated test certificates Safety: Hardware watchdog timer, emergency stop circuit, optocoupler isolation	IIoT-enabled remote test monitoring, Cloud-based compliance certificate blockchain, Digital Twin of MCB breaking mechanism, AI-based pass/fail waveform classification, Automated robotic MCB loading system

PS No.	Organization	Cat.	Problem Statement	Standard Tech Stack (Competition-Winning)	Advanced / Buzzword Tech
SIH25065	Ministry of Jal Shakti	Software	Designing and development of an application for on spot assessment of Roof Top Rain water harvesting and artificial recharge potential and size of the RTRWH and AR.	Frontend: React Native / Flutter (cross-platform mobile), HTML5, CSS3, JavaScript Backend: Python (Django / FastAPI), Node.js (Express) as alternative Database: PostgreSQL + PostGIS (geospatial queries), MongoDB for flexible document storage GIS/Mapping: Leaflet.js / Mapbox GL JS, OpenStreetMap tiles, Google Maps API, QGIS for offline geospatial analysis Hydrology Calculations: Python (NumPy, SciPy) — runoff coefficient models, rational method (Q=CIA), storage tank sizing algorithms, infiltration rate estimation Data Sources: IMD rainfall data API, CGWB aquifer maps, SRTM/DEM elevation data, Bhuvan (ISRO) satellite imagery API Camera/AR: Device camera for roof measurement (ARCore/ARKit for area estimation) Reports: PDF generation (jsPDF / ReportLab), offline-first with PWA capabilities Auth: Firebase Auth / JWT	LiDAR-based rooftop area computation via smartphone, Satellite-AI roof segmentation (DeepLabV3+), Digital Twin Water Balance Model, Blockchain-verified rainwater credits, GeoAI for automated aquifer recharge suitability mapping
SIH25066	Ministry of Jal Shakti	Software	Development of an AI-driven ChatBOT for INGRES as a virtual assistant	Frontend: React.js, HTML5, CSS3, JavaScript, WebSocket for real-time chat Backend: Python (FastAPI / Flask), Node.js (Express) as alternative NLP/AI: LangChain + RAG (Retrieval-Augmented Generation), OpenAI API / Gemini API / local LLM (Llama 3, Mistral via Ollama), sentence-transformers for embeddings, Bhashini API for multilingual support (Hindi + regional languages) Vector Database: ChromaDB / FAISS / Pinecone for document embedding search Knowledge Base: INGRES documents ingested via LlamaIndex / LangChain document loaders, PDF/DOCX parsing (PyMuPDF, python-docx) Database: PostgreSQL (conversation logs), Redis (session cache) Voice: Web Speech API / Google Speech-to-Text for voice input Deployment: Docker, Nginx, Unicorn, AWS/GCP or NIC Cloud Testing: pytest, Postman for API testing	Agentic RAG with tool-calling capabilities, Multi-modal input (image + text), Knowledge Graph integration (Neo4j) for connected water resource data, Conversational Memory with sliding window + summary buffers, Fine-tuned IndicBERT for groundwater domain-specific NLU
SIH25077	Government of Kerala	Hardware	A hardware that can detect and prevent unauthorized use of electric fences	Hardware: ESP32/STM32 MCU, current transformer (CT) sensors for line monitoring, voltage sensors (ZMPT101B), relay modules for fence power cutoff, GSM/4G module (SIM800L/SIM7600) for alerts, GPS module, tamper-proof enclosure (IP65), backup battery (18650 LiPo) Sensing: Non-invasive AC current sensor (SCT-013), pulse frequency analysis for fence energizer signature detection, PIR/IR beam sensors for perimeter Firmware: C/C++ (Arduino / ESP-IDF), FreeRTOS for multi-task scheduling Signal Processing: FFT-based frequency analysis to distinguish authorized vs unauthorized energizer patterns Backend: Python (Flask/FastAPI), MySQL/PostgreSQL Dashboard: React.js / HTML5+JS, Leaflet.js for location mapping Alerts: SMS (Twilio/GSM), push notifications (Firebase FCM), buzzer/siren Power: Solar panel + charge controller for off-grid deployment	AI-based energizer signature fingerprinting, LoRaMesh network for wide-area fence monitoring, Drone-based aerial fence inspection integration, Blockchain-based fence registration ledger, Computer Vision for physical fence tampering detection
SIH25080	Government of Kerala	Software	Document Overload at Kochi Metro Rail Limited (KMRL)-An automated solution	Frontend: React.js / Angular, HTML5, CSS3, JavaScript, Material UI Backend: Python (Django / FastAPI), Node.js (Express) Database: PostgreSQL (metadata), Elasticsearch (full-text search & indexing) Document Processing: Apache Tika / PyMuPDF for text extraction from PDFs, python-docx for DOCX, Tesseract OCR for scanned documents, pdf2image + PIL for image preprocessing NLP/AI: spaCy / NLTK for NER and keyword extraction, sentence-transformers for semantic search, topic modeling (LDA via Gensim), auto-summarization (T5/BART) Classification: scikit-learn / XGBoost for document categorization, rule-based tagging engine Storage: MinIO / AWS S3 for document blob storage, versioning support Workflow: Camunda / custom workflow engine for approval routing Auth: LDAP/AD integration, RBAC, JWT Deployment: Docker, Docker Compose, Nginx	LLM-powered intelligent document assistant (chat with your documents), Knowledge Graph construction from unstructured documents, Zero-shot document classification using CLIP/BERT, Robotic Process Automation (RPA) for auto-filing, Generative AI-based report drafting from ingested documents
SIH25081	Government of Kerala	Software	AI-Driven Train Induction Planning & Scheduling for Kochi Metro Rail Limited (KMRL)	Frontend: React.js, HTML5, CSS3, JavaScript, FullCalendar.js for scheduling UI, D3.js / Plotly for Gantt charts and visualizations Backend: Python (Django / FastAPI), Celery + Redis for async scheduling jobs Database: PostgreSQL (schedules, constraints, rolling stock data) Optimization: Google OR-Tools (constraint programming, vehicle routing), PuLP / SciPy.optimize for linear programming, custom genetic algorithm (DEAP library) for multi-objective scheduling AI/ML: scikit-learn / XGBoost for demand prediction, LSTM / Prophet for passenger flow forecasting, pandas, NumPy for data processing Simulation: SimPy for discrete-event train movement simulation Integration: REST APIs for real-time train status feeds, GTFS format support for public transit data Deployment: Docker, PostgreSQL replication, Nginx, Unicorn	Deep Reinforcement Learning (DRL) for dynamic rescheduling, Digital Twin of metro network for what-if scenario simulation, Quantum-inspired optimization (D-Wave / Qiskit), Multi-Agent System for decentralized train coordination, Federated Learning across metro systems for transfer-learning schedules

PS No.	Organization	Cat.	Problem Statement	Standard Tech Stack (Competition-Winning)	Advanced / Buzzword Tech
SIH25091	Government of Jammu and Kashmir	Software	AI-Based Timetable Generation System aligned with NEP 2020 for Multidisciplinary Education Structures	Frontend: React.js / Next.js, HTML5, CSS3, JavaScript, react-big-calendar / FullCalendar.js for timetable display Backend: Python (Django / FastAPI), Node.js (Express) as alternative Database: PostgreSQL (faculty, rooms, courses, constraints), Redis (caching) Optimization Engine: Google OR-Tools (CP-SAT solver for constraint satisfaction), PuLP for integer linear programming, custom genetic algorithm (DEAP / PyGAD) for timetable optimization NEP 2020 Logic: Multi-entry/multi-exit rules engine, credit-based semester mapping, major/minor/elective slot allocation logic AI/ML: scikit-learn for preference learning from historical timetable data, reinforcement learning (Stable Baselines3) for iterative improvement Export: PDF (ReportLab), Excel (openpyxl), iCal (.ics) format Auth: JWT, role-based access (Admin, HOD, Faculty, Student) Deployment: Docker, Nginx, Gunicorn, CI/CD with GitHub Actions	Graph Coloring Neural Networks for conflict-free scheduling, Transformer-based preference modeling for faculty satisfaction, Quantum Annealing for NP-hard timetable optimization, Self-adaptive evolutionary algorithms with real-time constraint relaxation, LLM-powered natural language constraint specification
SIH25102	Government of Rajasthan	Software	AI-based drop-out prediction and counseling system	Frontend: React.js / Next.js, HTML5, CSS3, JavaScript, Chart.js for analytics dashboards Backend: Python (Django / FastAPI), Celery for batch prediction jobs Database: PostgreSQL (student records, attendance, grades), Redis (caching) AI/ML: scikit-learn (Random Forest, Gradient Boosting), XGBoost / LightGBM for dropout risk scoring, pandas + NumPy for feature engineering, SMOTE for handling class imbalance Feature Engineering: Attendance %, grade trends, socioeconomic indicators, engagement metrics, distance-from-school, family income proxy Explainability: SHAP / LIME for transparent risk factor reporting Counseling Module: Appointment scheduling (FullCalendar.js), video call integration (Jitsi Meet / WebRTC), chatbot for student queries Notifications: SMS (Twilio / MSG91), Email (SendGrid), Push (Firebase FCM) Auth: JWT, RBAC (Admin, Teacher, Counselor, Student, Parent) Deployment: Docker, Nginx, Gunicorn	Temporal Graph Networks for social influence modeling on dropout, Survival Analysis (Cox Proportional Hazards) for time-to-dropout prediction, LLM-powered empathetic counseling chatbot, Causal Inference (DoWhy) for intervention impact estimation, Federated Learning across school districts for privacy-preserving model training
SIH25103	Government of Rajasthan	Software	ERP-based Integrated Student Management system	Frontend: React.js / Angular, HTML5, CSS3, JavaScript, Ant Design / Material UI Backend: Python (Django) or Java (Spring Boot), REST API architecture Database: PostgreSQL (primary RDBMS), Redis (session management, caching) ERP Modules: Admission management, fee collection (Razorpay/PayU integration), attendance tracking, exam & grading system, library management, hostel management, transport tracking, HR/payroll Reporting: Jasper Reports / ReportLab for PDF generation, Apache POI / openpyxl for Excel exports, Chart.js for dashboards Authentication: JWT + OAuth 2.0, LDAP integration, SSO support File Storage: MinIO / AWS S3 for documents and certificates Messaging: RabbitMQ / Kafka for inter-module event-driven communication Mobile: React Native / Flutter for companion app Deployment: Docker, Kubernetes (for scale), Nginx, PostgreSQL replication	Microservices Architecture with API Gateway (Kong/Traefik), Event Sourcing + CQRS for audit-grade data integrity, AI-powered academic analytics and performance prediction, Blockchain-based tamper-proof transcript/certificate issuance, Low-code workflow builder for custom administrative processes

SIH 2024 (55 Statements)

PS No.	Organization	Cat.	Problem Statement	Standard Tech Stack (Competition-Winning)	Advanced / Buzzword Tech
SIH1771	Ministry of Jal Shakti	Hardw are	Personalized testing kits for testing Residual Silver ion level at delivery points	Hardware: Colorimetric test strip with silver-ion-specific reagent (DPD analog for Ag+), portable spectrophotometer/colorimeter (photodiode + LED at specific wavelength), Arduino Nano / ESP32 for ADC reading and BLE data transfer, 3D-printed cuvette holder, LCD display (SSD1306 OLED) Optics: 420nm LED source, TSL2591 light sensor or OPT3001 for absorbance measurement Calibration: Known silver-ion standard solutions, polynomial curve fitting (NumPy) Mobile App: Flutter / React Native for result display, camera-based colorimetric analysis (OpenCV) Backend: Python (FastAPI), PostgreSQL for test logs, geolocation tagging Reports: Auto-generated PDF compliance reports	Electrochemical Ion-Selective Electrode (ISE) with nanomaterial-enhanced sensitivity, AI-based color gradient analysis for sub-ppm detection, IoT-connected water quality mesh network, Lateral Flow Assay with smartphone-readable QR-coded results
SIH1770	Ministry of Jal Shakti	Hardw are	Personalized testing kits for testing Bacteriological contamination at delivery points	Hardware: ATP bioluminescence sensor module (luminometer), enzyme substrate test (Colilert-type ONPG/MUG reagent), UV LED (365nm) + photodetector, ESP32/Arduino Nano MCU, temperature-controlled incubation chamber (Peltier + PID control), 3D-printed portable enclosure, OLED display Detection: Fluorescence detection circuit (PMT or photodiode), membrane filtration assembly for concentration Mobile App: Flutter / React Native, camera for fluoescence image capture, OpenCV for intensity quantification Backend: Python (FastAPI), PostgreSQL, GPS-tagged test results Compliance: IS 10500 / WHO guidelines comparison engine Connectivity: BLE for data sync to phone, 4G upload to central server	CRISPR-Cas12a/13 based rapid pathogen detection, Microfluidic Lab-on-a-Chip for single-cell bacterial counting, AI-powered colony morphology classification, Blockchain-immutable water safety certification chain
SIH1752	Ministry of Communication	Softwa re	Measurement and Monitoring of Counter Services - Develop a simple to use application for measurement and monitoring of counter services offered by Post Offices for their customers while providing ease of access to mail, parcel, financial, payment, insurance and citizen centric services from each Post Office.	Frontend: React.js / Next.js, HTML5, CSS3, JavaScript, responsive design (Bootstrap/MUI) Backend: Python (Django / FastAPI) or Node.js (Express) Database: PostgreSQL (transactional data), InfluxDB/TimescaleDB (time-series metrics) Computer Vision: YOLOv8 for people counting, OpenCV for queue length estimation, DeepSORT for person tracking in CCTV feed Video Processing: FFmpeg for stream handling, GStreamer, RTSP protocol Analytics: Python (pandas, Matplotlib, Plotly), service time calculation, wait time estimation, counter utilization metrics Dashboard: Grafana / custom React dashboard with real-time WebSocket updates Notifications: Email (SMTP), SMS (MSG91), push notifications Deployment: Docker, Nginx, CUDA-enabled GPU server for CV inference	Transformer-based activity recognition for service categorization, Emotion AI (sentiment analysis from facial cues) for customer satisfaction scoring, Digital Twin of post office layout for footfall simulation, Edge AI on NVIDIA Jetson for real-time on-premise processing
SIH1742	Ministry of Education	Softwa re	Standardizing Odd School Structures to Improve Educational Policy Implementation and Resource Allocation	Frontend: React.js, HTML5, CSS3, JavaScript, Ant Design / Material UI Backend: Python (Django / FastAPI), Node.js (Express) Database: PostgreSQL (school records, UDISE+ data), Elasticsearch (search/filter) Data Processing: pandas, NumPy, OpenPyXL for UDISE+ data ingestion, fuzzy matching (fuzzywuzzy / RapidFuzz) for school name deduplication Classification: scikit-learn (decision trees, clustering) for school structure categorization, rule-based engine for NEP 2020 structure mapping (5+3+3+4) GIS/Mapping: Leaflet.js / Mapbox, PostGIS for geospatial school distribution analysis Reporting: Chart.js / D3.js for policy dashboards, ReportLab for PDF reports Auth: JWT, RBAC (Central, State, District, Block levels) Deployment: Docker, Nginx, Unicorn	Graph Database (Neo4j) for school network relationship modeling, AI-powered resource optimization via multi-objective genetic algorithms, Predictive modeling for enrollment trend analysis, NLP-based policy document parsing for automated compliance checking
SIH1740	Indian Space Research Organization (ISRO)	Softwa re	Development of map-matching algorithm using AI-ML techniques to distinguish vehicular movement on highway and service road	Frontend: React.js, Leaflet.js / Mapbox GL JS for map visualization Backend: Python (FastAPI / Flask) GIS: PostGIS, OpenStreetMap data (Overpass API), OSRM for routing, Shapely / GeoPandas for geometric operations Map Matching: Hidden Markov Model (HMM) based map matching (Valhalla / GraphHopper), custom HMM implementation (hmmlearn), Viterbi algorithm AI/ML: scikit-learn (SVM, Random Forest for road classification), TensorFlow/PyTorch for deep learning approach (1D-CNN on GPS sequences), feature engineering: speed, heading change, altitude delta, HDOP Data: GPS trajectory datasets, road network shapefiles (NRSC/Bhuvan), pandas, NumPy for processing Evaluation: Precision/recall on ground-truth labeled trajectories Deployment: Docker, REST API for real-time map matching	Attention-based Sequence-to-Sequence model for trajectory classification, Graph Neural Networks on road network topology, Satellite imagery fusion (Sentinel-2) for road segment verification, Reinforcement Learning for adaptive map matching under GPS drift

PS No.	Organization	Cat.	Problem Statement	Standard Tech Stack (Competition-Winning)	Advanced / Buzzword Tech
SIH1737	Indian Space Research Organization (ISRO)	Software	Automatic Modulation Recognition software for DVB-S2X waveforms	Signal Processing: Python (NumPy, SciPy), GNU Radio for SDR pipeline, FFT/STFT for spectral analysis, cyclostationary feature extraction AI/ML: PyTorch / TensorFlow for deep learning AMR models, CNN (AlexNet/ResNet on constellation diagrams), LSTM/GRU for temporal feature extraction from I/Q samples, scikit-learn baseline (SVM, k-NN on cumulant features) DVB-S2X Specifics: LDPC/BCH decoder awareness, MODCOD identification, support for QPSK to 256APSK modulation schemes Data Generation: MATLAB/Simulink or GNU Radio for synthetic waveform generation with varying SNR, channel impairments (AWGN, Rician fading) GUI: PyQt5 / Tkinter for real-time spectrum and constellation display SDR Hardware: RTL-SDR / ADALM-Pluto / USRP B200 for real-world testing Deployment: C++ optimized inference, ONNX Runtime	Transformer-based AMR (Signal Transformer), Generative Adversarial Networks for data augmentation under low-SNR, Federated Learning across ground stations, Neuromorphic computing for ultra-low-latency modulation recognition, Explainable AI for modulation decision confidence scoring
SIH1736	Indian Space Research Organization (ISRO)	Software	AI based frame interpolation, video generation and display system for WMS services	Frontend: React.js, Leaflet.js / OpenLayers for WMS tile display, HTML5 Canvas/WebGL Backend: Python (FastAPI / Flask), GeoServer for WMS endpoints AI/Frame Interpolation: PyTorch (RIFE / IFRNet / AMT for frame interpolation), optical flow estimation (RAFT), temporal super-resolution Video Generation: FFmpeg for encoding/muxing, H.264/H.265 codec, OpenCV for frame manipulation Geospatial: GDAL, Rasterio for GeoTIFF processing, pyproj for CRS transformations, OGC WMS/WMTS protocol compliance Data: Satellite imagery tiles (Bhuvan, Sentinel), time-series raster data for weather/environment GPU: CUDA, cuDNN for accelerated inference Deployment: Docker, Nginx, GPU-enabled server (NVIDIA T4/A10)	Diffusion Models for satellite image temporal interpolation, Neural Radiance Fields (NeRF) for 3D geospatial view synthesis, Foundation Models for remote sensing (Prithvi by NASA/IBM), Streaming latent video generation for real-time WMS animation
SIH1735	Indian Space Research Organization (ISRO)	Software	On-device semantic segmentation of WMS services with geospatial data export	AI/ML: PyTorch / TensorFlow for training (DeepLabV3+, U-Net, SegFormer), ONNX export for cross-platform inference, TensorFlow Lite / ONNX Runtime Mobile for on-device deployment Dataset: Satellite imagery (Sentinel-2, Landsat), land use/land cover labels, augmentation (Albumentations) On-Device: Android (Kotlin/Java + TFLite), iOS (Swift + Core ML), model quantization (INT8) for mobile performance Geospatial: GDAL, Rasterio, GeoPandas for vector export (GeoJSON, Shapefile, KML), pyproj for coordinate transformations, WMS/WMTS client (OWSLib) Frontend: React Native / Flutter for cross-platform mobile app, Mapbox/Leaflet Export: GeoJSON, GeoTIFF with segmentation mask overlay, KML for Google Earth Backend: Python (FastAPI), PostgreSQL + PostGIS	Foundation Models for Earth Observation (Prithvi, SatMAE), Edge TPU / Neural Processing Unit acceleration, Self-supervised pre-training on unlabeled satellite imagery, Federated Learning for privacy-preserving distributed model training across devices
SIH1730	AICTE	Software	AI driven Inspection of Institutions	Frontend: React.js / Next.js, HTML5, CSS3, JavaScript, Material UI Backend: Python (Django / FastAPI), Node.js (Express) Database: PostgreSQL, Elasticsearch for document search Document Analysis: Tesseract OCR, PyMuPDF for PDF parsing, spaCy for NER (extract faculty names, infrastructure details), regex-based compliance checkers Computer Vision: YOLOv8 for infrastructure verification from uploaded images (lab equipment, classrooms), image classification (ResNet/EfficientNet) AI/ML: scikit-learn for risk scoring and anomaly detection, XGBoost for compliance prediction, NLP for self-assessment report analysis Video: WebRTC for remote virtual inspection, screen recording Scheduling: Celery + Redis for inspection task queue Reporting: ReportLab / WeasyPrint for auto-generated inspection reports Auth: JWT, OAuth 2.0, RBAC	LLM-powered automated compliance Q&A; from uploaded documents, 360° virtual campus tour verification using VR, Deepfake detection for video inspection integrity, Satellite imagery verification of campus area claims, AI-generated inspection report narratives
SIH1729	AICTE	Software	AI supported AICTE Approval process portal	Frontend: React.js / Angular, HTML5, CSS3, JavaScript, Ant Design Backend: Python (Django / FastAPI), Celery for background processing Database: PostgreSQL (application data), Redis (caching, session), Elasticsearch (search across applications) Document Processing: Tesseract OCR, PyMuPDF, python-docx, tabula-py for table extraction from PDFs AI/ML: scikit-learn (decision tree, rule-based) for auto-verification of criteria, NLP (spaCy) for document information extraction, anomaly detection for fraudulent submissions Workflow Engine: Django-viewflow / custom state machine for multi-stage approval Notifications: Email (Django email), SMS (MSG91), dashboard alerts Reporting: ReportLab, openpyxl for Excel exports Auth: JWT, OAuth 2.0, Aadhaar eKYC integration Deployment: Docker, Nginx, Unicorn, CI/CD	LLM-based intelligent form-filling assistant, Knowledge Graph for regulatory cross-referencing, AI-driven predictive application outcome scoring, Blockchain-based immutable approval audit trail, Robotic Process Automation for data entry from legacy systems

PS No.	Organization	Cat.	Problem Statement	Standard Tech Stack (Competition-Winning)	Advanced / Buzzword Tech
SIH1726	Ministry of Housing and Urban Affairs	Software	Utilization of aerial/drone-based images for monitoring of progress of construction activities for building construction projects	Frontend: React.js, Leaflet.js / Cesium.js (3D), HTML5, CSS3, JavaScript Backend: Python (Django / FastAPI) Computer Vision: YOLOv8 / Detectron2 for object detection (cranes, structures, materials), semantic segmentation (DeepLabV3+) for construction area mapping, OpenCV for image preprocessing Photogrammetry: OpenDroneMap (ODM) for orthomosaic and 3D point cloud generation, COLMAP for Structure-from-Motion Change Detection: Siamese CNN for temporal image comparison, pixel-level differencing with thresholding GIS: PostGIS, GDAL, Rasterio, GeoJSON overlays Storage: MinIO / AWS S3 for drone image storage Dashboard: Chart.js, Plotly for progress % visualization Database: PostgreSQL Deployment: Docker, GPU server for inference	4D BIM + Drone imagery overlay for as-planned vs as-built analysis, NeRF for photorealistic 3D construction site reconstruction, Foundation models (SAM - Segment Anything) for zero-shot construction element detection, Autonomous drone flight planning with AI-optimized waypoints
SIH1725	Ministry of Housing and Urban Affairs	Software	Utilization of images for monitoring of progress of construction activities for building construction projects.	Frontend: React.js, HTML5, CSS3, JavaScript, image comparison slider Backend: Python (Django / FastAPI) Computer Vision: YOLOv8 for construction element detection, OpenCV for image registration and alignment, ResNet/EfficientNet for progress stage classification Change Detection: Image differencing (structural similarity - SSIM), Siamese networks for temporal comparison Progress Estimation: Rule-based stage mapping (foundation→structure→finishing), regression models for % completion estimation Database: PostgreSQL, MinIO for image storage Mobile: React Native / Flutter for on-site photo capture with GPS tagging Reporting: Chart.js, ReportLab for progress reports Deployment: Docker, Nginx	Vision-Language Model (GPT-4V / Gemini Vision) for natural language construction status reports, BIM (Building Information Modeling) integration for planned vs actual overlay, Generative AI for construction milestone prediction, AR overlay on-site for expected vs current structure comparison
SIH1721	Ministry of Youth Affairs & Sports	Software	Intelligence and Investigations - Enhancing Anti-Doping Efforts	Frontend: React.js / Next.js, HTML5, CSS3, JavaScript, D3.js for network graphs Backend: Python (Django / FastAPI), Node.js Database: PostgreSQL (athlete profiles, test records), Neo4j (relationship graphs between athletes/coaches/suppliers) AI/ML: scikit-learn (anomaly detection - Isolation Forest), social network analysis (NetworkX) for suspicious connection detection, NLP (spaCy) for tip-off text analysis, pattern mining from ABP data Data Integration: ADAMS (Anti-Doping Administration & Management System) API integration, WADA code compliance rules engine Analytics: pandas, Matplotlib, Plotly for trend analysis, risk scoring algorithms for targeted testing Security: AES-256 encryption, RBAC, audit logging, GDPR/DPDP compliance Notifications: Secure email, encrypted messaging Deployment: Docker, air-gapped option for sensitive data	Graph Neural Networks for doping ring detection, Predictive AI for Biological Passport anomaly forecasting, NLP-powered dark web/social media monitoring for substance trafficking, Blockchain-based tamper-proof chain of custody for samples, Privacy-preserving computation (homomorphic encryption) for cross-border data sharing
SIH1711	Ministry of Railway	Software	Enhancing Rail Madad with AI-powered Complaint Management	Frontend: React.js / React Native (mobile), HTML5, CSS3, JavaScript Backend: Python (Django / FastAPI), Node.js (Express) Database: PostgreSQL (complaints), Elasticsearch (full-text search), Redis (caching) NLP/AI: spaCy / NLTK for text classification and entity extraction, BERT/IndicBERT for multilingual complaint categorization, sentiment analysis (VADER / transformers), Bhashini API for Hindi + regional language support Routing: Rule-based + ML (Random Forest) for auto-assignment to departments, priority scoring algorithm based on urgency keywords Analytics: pandas, Matplotlib, Plotly for complaint trends and resolution dashboards Image/Video: OpenCV + YOLOv8 for complaint evidence analysis (dirty coaches, broken fixtures) Notifications: SMS (MSG91), push (FCM), email Deployment: Docker, Nginx, Unicorn, CI/CD	LLM-powered auto-response drafting for common complaints, Voice-based complaint lodging with Whisper ASR, Emotion-aware priority escalation using sentiment transformers, Predictive maintenance triggers from complaint pattern analysis, Gamified resolution tracking for staff with leaderboard
SIH1709	Ministry of Power	Software	Comprehensive Automated Document Verification System for Official Documentation	Frontend: React.js, HTML5, CSS3, JavaScript, drag-and-drop file upload Backend: Python (Django / FastAPI) OCR: Tesseract OCR, PaddleOCR (better for multi-language), Google Vision API as fallback Document Parsing: PyMuPDF, pdf2image, python-docx, tabula-py for tables NLP/AI: spaCy (NER for names, dates, IDs), regex for pattern matching (PAN, Aadhaar, GSTIN formats), LayoutLM / LayoutLMv3 for document understanding Verification: Cross-referencing with DigiLocker API, hash-based document tampering detection (SHA-256), digital signature validation (PyKCS11) Database: PostgreSQL, MinIO for document storage Security: AES-256 encryption at rest, TLS 1.3, audit trail logging Deployment: Docker, Nginx	Zero-Knowledge Proof for privacy-preserving document verification, Deepfake/forgery detection using forensic AI, Verifiable Credentials (W3C standard) with DID (Decentralized Identifiers), Multi-modal document authenticity scoring using transformer ensembles

PS No.	Organization	Cat.	Problem Statement	Standard Tech Stack (Competition-Winning)	Advanced / Buzzword Tech
SIH1702	Ministry of Law & Justice	Software	Bail Reckoner	Frontend: React.js / Next.js, HTML5, CSS3, JavaScript, step-by-step wizard UI for data input Backend: Python (Django / FastAPI) Database: PostgreSQL (case data, bail criteria, IPC/BNS sections) Legal Logic: Rule-based engine mapping IPC/BNS sections to bail provisions, decision tree for bailable/non-bailable/cognizable classification, CrPC Section 436/437/439 logic implementation NLP: spaCy for legal entity extraction, sentence-transformers for case similarity matching (precedent search), LangChain + RAG for AI-assisted bail argument research Data: Indian Kanoon API / eSCR for case law database, structured IPC/BNS/CrPC dataset Reporting: ReportLab for bail application draft generation Auth: JWT, RBAC (Judge, Lawyer, Court Staff) Deployment: Docker, Nginx, data encryption at rest	Legal LLM fine-tuned on Indian case law (InLegalBERT), Graph-based case precedent network analysis, Predictive bail outcome scoring using historical data, Blockchain-based immutable bail order registry, AI-generated bail risk assessment reports with explainability
SIH1701	Ministry of Law & Justice	Software	AI-Driven Research Engine for Commercial Courts	Frontend: React.js / Next.js, HTML5, CSS3, JavaScript, faceted search UI, document viewer (PDF.js) Backend: Python (Django / FastAPI), Elasticsearch for legal document search Database: PostgreSQL (case metadata), Elasticsearch (full-text indexing) NLP/AI: sentence-transformers for semantic search, LangChain + RAG (ChromaDB/FAISS as vector store), spaCy + custom NER for legal entities (parties, sections, citations), text summarization (BART / T5), citation network analysis (NetworkX) Legal Data: Indian Kanoon scraper/API, eSCR database, Commercial Courts Act provisions Features: Case timeline generation, related judgment clustering, argument extraction, statutory provision linking Auth: JWT, RBAC Deployment: Docker, Elasticsearch cluster, Nginx	Legal-domain Large Language Model (LegalLLM / InLegalBERT), Knowledge Graph of Indian commercial law with ontology (OWL/RDF), Cross-lingual legal research (Hindi-English bilingual models), Automated legal brief generation using generative AI, Quantum-enhanced similarity search for massive case databases
SIH1700	Ministry of Law & Justice	Software	Developing an AI based interactive Chatbot or virtual assistant for the Department of Justice's Website.	Frontend: React.js, HTML5, CSS3, JavaScript, chat widget (Botpress UI / custom) Backend: Python (FastAPI / Flask), Node.js NLP/AI: LangChain + RAG with FAISS/ChromaDB, OpenAI API / Gemini API / local Llama 3 (via Ollama), Bhashini API for multilingual (Hindi + 12 regional languages), Whisper for voice-to-text Knowledge Base: DoJ website content, legal FAQs, Acts & Rules, LlamaIndex document loaders for PDF/HTML ingestion Database: PostgreSQL (logs), Redis (sessions), vector DB (ChromaDB) Voice: Web Speech API, Google TTS for voice responses Analytics: Query analytics dashboard, feedback collection, unanswered query tracking Auth: Session-based, optional login for personalized history Deployment: Docker, Nginx, Gunicorn	Multi-turn conversational AI with memory (ConversationBufferMemory), Agentic RAG with tool-use for live case status lookup, Emotion-aware response generation, Avatarized AI assistant with lip-sync (SadTalker/Wav2Lip), Voice cloning for consistent assistant persona
SIH1697	Ministry of Jal Shakti	Software	AI based chatbot for collating and dissemination of information on groundwater.	Frontend: React.js, HTML5, CSS3, JavaScript, chat UI component Backend: Python (FastAPI / Flask) NLP/AI: LangChain + RAG, FAISS / ChromaDB for vector search, sentence-transformers for embeddings, OpenAI API / Gemini API / Llama 3 for response generation, Bhashini API for multilingual support Knowledge Base: CGWB reports, aquifer maps, DWLR data, well inventory databases, PDF/CSV ingestion pipeline GIS Integration: Leaflet.js for groundwater level maps, PostGIS for spatial queries, CGWB GIS data layers Database: PostgreSQL + PostGIS, Redis Data Processing: pandas, NumPy for water level trend analysis Deployment: Docker, Nginx, Gunicorn	Multimodal chatbot (accept map queries, images of well logs), Knowledge Graph of aquifer systems (Neo4j), GeoAI for groundwater potential zone prediction, LLM fine-tuned on hydrogeology corpus, Real-time DWLR data streaming into chatbot responses
SIH1696	Ministry of Jal Shakti	Software	A software application - Ground Water Level Predictor	Frontend: React.js, HTML5, CSS3, JavaScript, Plotly.js / Chart.js for time-series plots, Leaflet.js for spatial visualization Backend: Python (Django / FastAPI) Database: PostgreSQL + PostGIS (well locations, historical levels), TimescaleDB for time-series AI/ML: LSTM / GRU (PyTorch/TensorFlow) for time-series forecasting, Prophet (Facebook) for seasonal decomposition, XGBoost / LightGBM for feature-based prediction, scikit-learn for baseline models Features: Rainfall (IMD data), temperature, soil type, land use, pumping rates, river proximity, previous water levels Data Sources: CGWB DWLR data, IMD rainfall API, ISRO Bhuvan land use Visualization: Heatmaps, contour plots (SciPy interpolation), prediction confidence intervals Export: CSV, PDF reports (ReportLab) Deployment: Docker, Nginx, scheduled model retraining (Celery/cron)	Transformer-based temporal forecasting (Temporal Fusion Transformer), Physics-Informed Neural Networks (PINNs) incorporating Darcy's law, Ensemble deep learning with uncertainty quantification, Satellite-derived soil moisture fusion (SMAP/Sentinel-1), AutoML (Auto-sklearn/TPOT) for automated model selection

PS No.	Organization	Cat.	Problem Statement	Standard Tech Stack (Competition-Winning)	Advanced / Buzzword Tech
SIH1695	Ministry of Jal Shakti	Software	A software application for analysis of DWLR data and raise alarms in respect of anomalous values, faulty DWLRs etc	Frontend: React.js, HTML5, CSS3, JavaScript, Grafana-style dashboards, Leaflet.js for DWLR location map Backend: Python (FastAPI / Django), Celery for scheduled anomaly checks Database: PostgreSQL + TimescaleDB (time-series DWLR data), Redis Anomaly Detection: scikit-learn (Isolation Forest, LOF, DBSCAN), statistical methods (Z-score, IQR, moving average deviation), LSTM-based anomaly detection for temporal patterns Fault Detection: Rule-based checks (stuck values, sudden jumps, flatline detection, value range violations per aquifer type), sensor health scoring algorithm Alerting: Email (SMTP), SMS (MSG91/Twilio), dashboard notifications, configurable alert thresholds per DWLR Reporting: Automated daily/weekly reports (ReportLab/Matplotlib), pandas for data aggregation Deployment: Docker, Nginx, cron/Celery for periodic analysis	AutoEncoder-based anomaly detection for complex failure patterns, Predictive maintenance of DWLR hardware using survival analysis, Digital Twin of well-DWLR system for simulation, Federated anomaly detection across state-level DWLR networks, Self-healing alerts with automated ticket creation (Jira/ServiceNow API)
SIH1694	Ministry of Jal Shakti	Software	Real-time Ganga river water quality forecasting using AI-enabled DSS, satellite data, IoT, and dynamic models.	Frontend: React.js, Leaflet.js / Mapbox for river stretch visualization, D3.js / Plotly.js for water quality dashboards Backend: Python (FastAPI / Django), Celery for async data ingestion Database: PostgreSQL + PostGIS + TimescaleDB, InfluxDB alternative IoT Integration: MQTT broker (Mosquitto), REST APIs for sensor data ingestion, CPCB real-time water quality API Satellite Data: Sentinel-2/Landsat (Google Earth Engine API), NDWI/chlorophyll-a indices, Rasterio/GDAL for raster processing AI/ML: LSTM/GRU for time-series WQ forecasting, Random Forest/XGBoost for feature-based prediction, QUAL2E/WASP-type water quality model integration DSS: Decision matrix for intervention recommendations, scenario simulation engine Parameters: BOD, COD, pH, DO, turbidity, fecal coliform, temperature Deployment: Docker, GPU server for model inference, CI/CD	Physics-Informed Neural Networks for hydrodynamic-water quality coupling, Digital Twin of Ganga river system, Satellite-AI fusion for real-time chlorophyll and turbidity mapping, Edge AI on IoT nodes for local anomaly detection, Transformer-based multi-variate water quality forecasting
SIH1693	Ministry of Jal Shakti	Software	Developing a Robust Hydraulic Transient Analysis Model for Hydro Power and Pumped Storage Schemes.	Numerical Methods: Python (NumPy, SciPy) - Method of Characteristics (MOC) for water hammer simulation, finite difference/finite volume schemes for pressure wave propagation Simulation: MATLAB/Simulink (industry standard for hydraulic transients), Python alternatives: PyFluids, EPANET (via wntr library) Physics: Joukowski equation implementation, surge tank modeling, turbine/pump characteristic curves (hill charts), governor response modeling Frontend: React.js, Plotly.js / D3.js for pressure-time curves, 3D pipeline visualization (Three.js) Backend: Python (FastAPI), job queue (Celery) for long simulations Database: PostgreSQL (project configurations, simulation results) Validation: Comparison with HAMMER/AFT Impulse benchmarks Export: CSV, PDF reports with pressure envelope diagrams	Physics-Informed Neural Networks (PINNs) for real-time transient prediction, Digital Twin of penstock-turbine system, GPU-accelerated CFD (OpenFOAM) for 3D transient validation, Reduced-Order Models (ROM) for real-time control system integration, ML-based surrogate models replacing expensive MOC simulations
SIH1688	Ministry of Jal Shakti	Software	Development of handheld device/Mobile based Operation & Maintenance tool for asset & consumables inventories and finance management in context of drinking water supply scheme.	Mobile: Flutter / React Native (cross-platform), offline-first with SQLite local DB Backend: Python (Django / FastAPI), Node.js (Express) Database: PostgreSQL (server), SQLite (mobile offline), sync via REST API + conflict resolution Features: QR/barcode scanner (ZXing library) for asset tracking, GPS tagging for asset location, camera for condition documentation, offline form submission with auto-sync Inventory: FIFO/LIFO tracking, reorder level alerts, stock ledger, consumable usage analytics (pandas) Finance: Budget tracking, expenditure recording, bill/voucher upload, simple accounting module (income-expense ledger) Reporting: Chart.js for mobile dashboards, ReportLab for PDF reports Auth: JWT, RBAC (State, District, Block, GP levels), biometric option Notifications: Push (FCM), SMS for low stock alerts Deployment: Docker, Nginx, Play Store / APK distribution	AR-based asset identification (point camera to auto-detect equipment), Predictive consumable demand forecasting using time-series ML, Blockchain-based transparent finance audit trail, NFC/RFID asset tagging for touchless inventory, AI-powered image-based asset condition assessment
SIH1686	National Technical Research Organisation (NTRO)	Software	Tools and techniques for customisation of GPO as per CIS guidelines to deploy on offline / standalone windows.	Core: PowerShell (primary scripting), Windows Group Policy Management (GPMC), LGPO.exe (Microsoft Local Group Policy Object utility), Security Compliance Toolkit (SCT) from Microsoft CIS Benchmarks: CIS-CAT Pro Assessor for validation, CIS Benchmark PDF parser (Python + PyMuPDF) for rule extraction Automation: PowerShell DSC (Desired State Configuration) for declarative policy enforcement, Ansible (Windows modules) for multi-machine deployment, Python (subprocess) for orchestration GUI: C# WPF / Electron.js for user-friendly GPO customization tool Registry: Direct registry manipulation (reg.exe, PowerShell Set-ItemProperty), SecurityPolicy .inf file generation Validation: Automated compliance scanning (CIS-CAT), custom PowerShell scripts for pre/post-deployment verification Packaging: NSIS / WIX for installer creation, USB-deployable offline package Logging: Windows Event Log, custom XML-based audit trail	Zero Trust Architecture alignment with CIS + NIST 800-53, ML-based policy conflict detection and resolution, Infrastructure-as-Code (IaC) for reproducible hardened OS images, Automated STIG compliance cross-mapping, AI-driven threat surface analysis post-GPO application

PS No.	Organization	Cat.	Problem Statement	Standard Tech Stack (Competition-Winning)	Advanced / Buzzword Tech
SIH1685	National Technical Research Organisation (NTRO)	Software	Building Offline Parallel AV Pipeline to cater multiple AVs for CII entities.	Core Architecture: Python / C++ orchestration layer for parallel AV execution, multi-process architecture (multiprocessing / subprocess), YARA rules engine for custom signature scanning AV Integration: ClamAV (open-source, offline capable), integration with multiple AV engines via command-line interfaces, VirusTotal-compatible offline hash database Scanning Pipeline: File queue manager (RabbitMQ / ZeroMQ for local IPC), hash-based deduplication (SHA-256), file type detection (libmagic / python-magic), archive extraction (7-Zip, unrar, cabextract) Signature Updates: Sneakernet-compatible update package (USB transfer), differential signature update mechanism, GPG-signed update verification Dashboard: Electron.js / PyQt5 desktop application, scan result aggregation, conflict resolution when AVs disagree Database: SQLite (local), scan history, quarantine management Deployment: NSIS/WiX installer, portable deployment option	ML-based malware classification (PE header analysis, opcode sequences), Sandbox integration (Cuckoo Sandbox) for behavioral analysis, MITRE ATT&CK; mapping for detected threats, Federated signature sharing between air-gapped CII nodes, Homomorphic encryption for secure multi-AV result aggregation
SIH1682	National Technical Research Organisation (NTRO)	Software	Centralized Automated Solution for Price Estimation & Reasonability.	Frontend: React.js / Angular, HTML5, CSS3, JavaScript, data tables (AG Grid / DataTables) Backend: Python (Django / FastAPI), Node.js Database: PostgreSQL (historical price data, vendor catalog), Elasticsearch (price search) Data Collection: Web scraping (Scrapy / BeautifulSoup / Playwright) for market price data, GeM API integration for government marketplace prices, structured data ingestion from vendor quotations (PDF parsing via Tabula/Camelot) Price Estimation: Statistical methods (mean, median, percentile analysis), regression models (scikit-learn), time-series (Prophet) for price trend forecasting, parametric cost estimation models Reasonability: IQR-based outlier detection, benchmark comparison, Should-Cost Model implementation, historical price adjustment (inflation indexing via WPI/CPI data) Reporting: Auto-generated price reasonability certificates, Chart.js for trend visualization, ReportLab for PDF Auth: JWT, RBAC, audit trail Deployment: Docker, Nginx	LLM-powered natural language price query interface, Graph-based vendor relationship analysis for cartel detection, Reinforcement Learning for dynamic price negotiation strategy, Blockchain-based transparent procurement price registry, AutoML for automated cost model selection and tuning
SIH1680	National Technical Research Organisation (NTRO)	Software	Few Shot Language Agnostic Key Word Spotting system (FSLAKWS) for audio files.	Audio Processing: Python (librosa, soundfile, torchaudio), FFmpeg for format conversion, VAD (Voice Activity Detection) - WebRTC VAD / Silero VAD Feature Extraction: Mel-spectrograms, MFCC, log-filterbanks, wav2vec 2.0 / HuBERT for self-supervised audio representations Few-Shot Learning: Prototypical Networks / Matching Networks (PyTorch), Siamese Networks for audio embedding similarity, metric learning with triplet loss Language Agnostic: Multilingual pre-trained models (XLS-R / MMS by Meta), dynamic time warping (DTW) for query-by-example matching Backend: Python (FastAPI), Celery for batch audio processing Database: PostgreSQL (metadata), MinIO/S3 for audio file storage GUI: React.js web interface for audio upload, keyword registration, search results Deployment: Docker, GPU server for inference, ONNX Runtime for optimized serving	Whisper-based zero-shot keyword detection via ASR transcript mining, Contrastive Learning (SimCLR) for robust audio embeddings, Neural Architecture Search for optimal KWS model, On-device KWS using TinyML for edge deployment, Attention-based keyword localization in long audio streams
SIH1672	Ministry of Electronics and Information Technology	Software	Develop a ML Model based solution to refine CAPTCHA.	Frontend: React.js, HTML5 Canvas, CSS3, JavaScript for CAPTCHA widget Backend: Python (FastAPI / Flask), Node.js CAPTCHA Generation: Python (Pillow, captcha library) for image generation, random distortions (noise, rotation, warping, line overlay), font randomization, color jittering Adversarial Testing: PyTorch / TensorFlow - CNN (ResNet, EfficientNet) trained to break CAPTCHAs, adversarial examples (FGSM, PGD) for robustness testing Behavioral Analysis: Mouse movement tracking (JS), typing pattern analysis, interaction timing features, scikit-learn classifier for bot vs human behavior Challenge Types: Image distortion CAPTCHA, puzzle CAPTCHA (drag-and-drop), audio CAPTCHA (gTTS + noise overlay), math CAPTCHA Security: Rate limiting, token-based validation, session binding Database: Redis (token store, rate limit), PostgreSQL (analytics) Deployment: Docker, CDN for static assets	GAN-generated adversarial CAPTCHAs that auto-evolve against bots, Proof-of-Work based invisible CAPTCHA, Multimodal CAPTCHA combining vision + audio + interaction, Federated Learning for continuous CAPTCHA difficulty calibration, Zero-Knowledge Proof CAPTCHA for privacy-preserving human verification
SIH1671	Ministry of Electronics and Information Technology	Software	Develop a functional solution that demonstrates the face liveness detection	Frontend: React.js, HTML5, CSS3, JavaScript, WebRTC for live camera feed Backend: Python (FastAPI / Flask) Face Detection: MediaPipe Face Mesh, dlib (68-point landmark), MTCNN, RetinaFace Liveness Detection: PyTorch / TensorFlow - binary classifier (live vs spoof), depth estimation from monocular RGB (MiDaS), texture analysis (LBP - Local Binary Patterns), blink detection + head movement tracking, challenge-response (turn head left/right, smile) Anti-Spoofing: Multi-frame analysis, frequency domain analysis (FFT on face patches), Moiré pattern detection for screen replay attacks, OULU-NPU / CASIA-FASD datasets for training Mobile: TensorFlow Lite / Core ML for on-device inference Database: PostgreSQL (session logs), MinIO for frame storage Deployment: Docker, ONNX Runtime for optimized serving	3D Morphable Model (3DMM) fitting for depth-based liveness verification, Remote Photoplethysmography (rPPG) - heartbeat detection from face video, Vision Transformer for domain-generalized anti-spoofing, Neural Radiance Fields for 3D face reconstruction to detect 2D attacks, Multimodal fusion (RGB + NIR + depth) for enterprise-grade liveness

PS No.	Organization	Cat.	Problem Statement	Standard Tech Stack (Competition-Winning)	Advanced / Buzzword Tech
SIH1670	Ministry of Electronics and Information Technology	Software	Develop a functional solution that incorporates the security of the ML model.	ML Framework: PyTorch / TensorFlow for model development Adversarial Defense: ART (Adversarial Robustness Toolbox by IBM), CleverHans library, adversarial training (PGD-AT), input validation and preprocessing defenses Model Protection: ONNX model encryption, model watermarking (embedding verification triggers), differential privacy (Opacus for PyTorch), model obfuscation and weight encryption Inference Security: Secure enclave deployment (Intel SGX / ARM TrustZone concepts), input sanitization pipeline, output confidence thresholding Monitoring: Model drift detection (Evidently AI / Alibi Detect), adversarial input detection (statistical outlier checks), logging and alerting on suspicious queries Backend: Python (FastAPI), Redis for rate limiting Frontend: React.js demo dashboard showing attack scenarios and defenses Deployment: Docker, model served via TorchServe / TF Serving, API gateway with auth	Homomorphic Encryption for privacy-preserving ML inference, Federated Learning for decentralized model training without data sharing, Zero Trust ML Pipeline with continuous integrity verification, Secure Multi-Party Computation (MPC) for collaborative model training, AI-based adversarial attack detection using meta-learning
SIH1669	Ministry of Electronics and Information Technology	Software	Transformo Docs Application: Empowering Machine-Readable Document Management System.	Frontend: React.js / Next.js, HTML5, CSS3, JavaScript, PDF.js for document viewer Backend: Python (Django / FastAPI), Node.js OCR: Tesseract OCR, PaddleOCR (for multi-language + layout-aware), EasyOCR for handwritten text Document AI: LayoutLMv3 / DocTR for structure-aware text extraction, table extraction (Camelot / Tabula), Apache Tika for multi-format parsing (PDF, DOCX, images, scans) NLP: spaCy for NER, text classification, keyword extraction, auto-tagging with TF-IDF / BERT embeddings Output Formats: JSON, XML, CSV, machine-readable PDF (PDF/A), structured data export via API Storage: MinIO / S3 for documents, Elasticsearch for full-text search Database: PostgreSQL (metadata, extracted fields) Auth: JWT, RBAC, audit logging Deployment: Docker, Nginx	Foundation Document Model (Donut / Nougat by Meta) for end-to-end document understanding, GPT-4V / Gemini Vision for visual document Q&A, Knowledge Graph auto-construction from extracted entities, Blockchain-based document provenance and integrity chain, Self-supervised pre-training on government document corpus
SIH1660	Ministry of Earth Sciences	Software	Interactive gamified approach to Ocean Literacy	Frontend: React.js, HTML5, CSS3, JavaScript, Phaser.js / PixiJS for 2D game engine, Three.js for 3D ocean visualization Backend: Node.js (Express) / Python (FastAPI) Database: MongoDB (flexible schema for game data) / PostgreSQL, Redis (leaderboard, session) Gamification: Points, badges, levels, leaderboards, quizzes, story-driven missions (ocean exploration scenarios), achievement system Content: INCOIS ocean data integration, educational modules, interactive maps (Leaflet.js), animated infographics (Lottie.js) Mobile: PWA (Progressive Web App) or React Native Audio/Visual: Howler.js for sound effects, CSS animations, SVG illustrations, particle effects Analytics: User engagement tracking, learning outcome assessment, Chart.js for educator dashboards Auth: Firebase Auth / JWT, social login Deployment: Vercel / Netlify (frontend), Docker (backend)	WebXR-based underwater VR exploration experience, AI-powered adaptive difficulty (Bayesian Knowledge Tracing), Procedural ocean environment generation using Wave Function Collapse, LLM-powered NPC ocean scientists for interactive Q&A, Metaverse-ready multiplayer ocean exploration
SIH1657	Ministry of Earth Sciences	Software	Integrated Geo-Referenced Fish Catch Data Repository and Access System	Frontend: React.js, Leaflet.js / Mapbox GL JS for geo-referenced visualization, HTML5, CSS3, JavaScript, D3.js for catch analytics Backend: Python (Django / FastAPI), Node.js Database: PostgreSQL + PostGIS (spatial queries on catch locations), Elasticsearch (search across species, zones, dates) Data Ingestion: REST APIs for field data upload, CSV/Excel bulk import, mobile app (Flutter/React Native) with GPS auto-tagging for fishermen GIS: GeoServer for WMS/WFS publishing, QGIS for data preparation, fishing zone boundary shapefiles, EEZ mapping Analytics: pandas, NumPy for catch trend analysis, species diversity indices, seasonal heatmaps, CPUE calculations Reporting: Chart.js, Plotly.js dashboards, ReportLab for PDF reports Auth: JWT, RBAC (Admin, Researcher, Fisher, Public), API keys for data access Deployment: Docker, Nginx, PostGIS replication	AI-powered species identification from catch photos (image classification), Satellite-derived ocean color data (chlorophyll-a) for catch prediction, Blockchain-based catch certification for traceability, Digital Twin of fishing zones with real-time vessel tracking (AIS), LLM-powered natural language query interface for catch data

PS No.	Organization	Cat.	Problem Statement	Standard Tech Stack (Competition-Winning)	Advanced / Buzzword Tech
SIH1655	Ministry of Earth Sciences	Software	Detecting oil spills at marine environment using Automatic Identification System (AIS) and satellite datasets	Frontend: React.js, Leaflet.js / Mapbox GL JS, HTML5, CSS3, JavaScript Backend: Python (FastAPI / Django) Satellite Processing: Google Earth Engine (GEE) API, Sentinel-1 SAR data (dark spot detection), SNAP (ESA) for SAR preprocessing, GDAL / Rasterio for raster ops AIS Data: AIS decoder (pyais library), vessel trajectory analysis, anomaly detection in vessel behavior (speed changes, route deviations) AI/ML: U-Net / DeepLabV3+ for SAR image segmentation (oil vs water), Random Forest / SVM for feature-based classification (texture, contrast, homogeneity), correlation model: AIS vessel proximity + SAR dark spots GIS: PostGIS for spatial joins (vessel tracks × spill polygons), GeoPandas, Shapely Database: PostgreSQL + PostGIS, TimescaleDB for AIS time-series Alerting: Email, SMS, dashboard alerts for detected spills Deployment: Docker, GPU server for SAR image inference	Foundation Models for SAR (Phileo / SSL4EO) for few-shot oil spill detection, Multi-temporal SAR change detection using attention mechanisms, Graph Neural Networks for vessel-spill attribution analysis, Real-time satellite tasking integration for rapid spill confirmation, Digital Twin of coastal marine environment
SIH1654	Ministry of Defence	Software	Determining expert relevance with respect to interview board subject and candidates' area of expertise	Frontend: React.js, HTML5, CSS3, JavaScript, Material UI Backend: Python (Django / FastAPI) Database: PostgreSQL (expert profiles, subject taxonomies, board schedules) NLP/AI: sentence-transformers for semantic similarity between expert profile and subject, TF-IDF + cosine similarity as baseline, spaCy for keyword extraction from CVs/profiles, BERT/IndicBERT for domain-aware text embeddings Matching: Hungarian Algorithm for optimal expert-board assignment, constraint satisfaction (Google OR-Tools) with availability/conflict constraints Knowledge Base: Subject taxonomy tree, expert publication/experience database Scoring: Weighted scoring model (experience years, publications, past board performance), scikit-learn for relevance ranking Reporting: Auto-generated panel composition reports (ReportLab) Auth: JWT, RBAC, encryption for sensitive data	Knowledge Graph of expert-subject relationships (Neo4j), LLM-powered expert profile summarization and gap analysis, Graph-based collaborative filtering for expert recommendation, Academic paper embedding (SciBERT/Specter) for research expertise mapping, Federated expert database across multiple organizations
SIH1653	Ministry of Defence	Software	Web based Selector-Applicant Simulation Software	Frontend: React.js / Next.js, HTML5, CSS3, JavaScript, drag-and-drop UI (react-dnd), real-time collaboration (Socket.io) Backend: Python (Django / FastAPI) / Node.js (Express), WebSocket server Database: PostgreSQL (applicant data, selector criteria, simulation configs) Simulation Engine: Monte Carlo simulation (NumPy/SciPy) for outcome modeling, configurable scoring rubrics, weighted criteria matrix, multi-round selection process simulation Analytics: pandas for data processing, Matplotlib/Plotly for outcome distributions, statistical tests (chi-square, t-test) for bias detection Features: Role-play scenarios, timed assessments, video interview simulation (WebRTC), automated scoring with manual override Reporting: Chart.js dashboards, ReportLab for PDF outcome reports Auth: JWT, RBAC (Selector, Applicant, Admin, Observer) Deployment: Docker, Nginx, WebSocket-compatible reverse proxy	AI-powered behavioral analysis from video interviews (emotion + confidence scoring), Digital Twin of selection process for what-if scenario analysis, LLM-based auto-evaluation of written responses, Gamified assessment with adaptive difficulty, VR-based immersive group task simulation
SIH1652	Ministry of Defence	Software	Extraction and Verification of Information from semi-categorised data.	Frontend: React.js, HTML5, CSS3, JavaScript, data table with filters (AG Grid) Backend: Python (Django / FastAPI) Data Extraction: Apache Tika / PyMuPDF for multi-format parsing, Tesseract OCR for scanned docs, regex + rule-based extraction for structured patterns, spaCy NER for entity extraction (names, dates, locations, IDs) Classification: scikit-learn (Naive Bayes, SVM) for category assignment, BERT/DistilBERT for text classification, semi-supervised learning (Label Propagation) for partially labeled data Verification: Cross-referencing extracted fields against master databases, consistency checks (date ranges, format validation), confidence scoring for extracted values Database: PostgreSQL, Elasticsearch for search Reporting: Discrepancy reports (ReportLab), data quality dashboards (Plotly) Auth: JWT, RBAC, audit trail Deployment: Docker, Nginx	LayoutLMv3 for visual document understanding, Active Learning for efficient labeling of ambiguous categories, Knowledge Graph for cross-document entity resolution, LLM-powered data cleaning and normalization, Automated data lineage tracking for provenance verification

PS No.	Organization	Cat.	Problem Statement	Standard Tech Stack (Competition-Winning)	Advanced / Buzzword Tech
SIH1646	Ministry of Coal	Software	App based Project monitoring of S&T; / R&D; Projects of Coal Sector	Mobile: Flutter / React Native (cross-platform), offline capability Frontend Web: React.js, HTML5, CSS3, JavaScript Backend: Python (Django / FastAPI), Node.js Database: PostgreSQL (project data, milestones, budgets), Redis Project Management: Gantt chart (Frappe Gantt / dhtmlxGantt), milestone tracking, fund utilization monitoring, document upload and versioning (MinIO) Analytics: pandas for project performance metrics (SPI, CPI), Chart.js / Plotly for dashboards, deviation alerts Workflow: Multi-level approval workflow (Django-viewflow), progress report submission and review cycle Notifications: Push (FCM), email, SMS for deadline reminders Reporting: ReportLab for auto-generated quarterly reports, Excel exports Auth: JWT, RBAC (Ministry, PI, Institution, Reviewer) Deployment: Docker, Nginx, Play Store / App Store	AI-powered project delay prediction and risk scoring, NLP-based automated progress report summarization, Blockchain-based fund flow tracking for transparency, Digital Twin of R&D; project lifecycle, LLM-powered Q&A; over project documents
SIH1645	Ministry of Coal	Software	An app and web based software for Productivity and safety management of coal mines.	Mobile: Flutter / React Native, offline-first architecture Frontend Web: React.js, HTML5, CSS3, JavaScript Backend: Python (Django / FastAPI), Node.js Database: PostgreSQL + TimescaleDB (sensor data), Redis Safety Module: Incident reporting forms, hazard identification checklists, DGMS compliance tracking, safety audit scheduling, near-miss reporting with photo/GPS evidence Productivity: OEE (Overall Equipment Effectiveness) calculator, shift-wise production logging, equipment downtime tracking, target vs actual dashboards IoT Integration: MQTT for real-time sensor data (gas levels, ventilation, ground stability), alert thresholds for CH4, CO, O2 levels Analytics: pandas, scikit-learn for accident pattern analysis, Chart.js / Plotly dashboards Notifications: SOS alerts (SMS, siren trigger), push notifications Deployment: Docker, Nginx, edge server for mine sites with poor connectivity	Computer Vision for PPE compliance detection (helmet, vest, boots), Predictive safety analytics using historical incident data, Digital Twin of mine for simulation and planning, AR-based safety training modules, Wearable IoT integration for miner health monitoring (heart rate, fatigue)
SIH1625	Government of NCT of Delhi	Software	Smart Classroom Management Software for Enhanced Learning Environments	Frontend: React.js / Next.js, HTML5, CSS3, JavaScript, Material UI Backend: Python (Django / FastAPI) / Node.js (Express) Database: PostgreSQL (student/teacher/class data), MongoDB (content), Redis Features: Attendance (QR code / face recognition), digital timetable, assignment submission portal, live quiz/poll (Socket.io for real-time), resource sharing (PDF, video, links) AI/ML: Engagement detection from webcam (facial landmark analysis - MediaPipe), attention scoring, personalized content recommendations (collaborative filtering) Video: WebRTC for live classes, HLS for recorded content Analytics: Student performance dashboards, attendance trends, learning outcome tracking, Chart.js / Plotly Mobile: React Native / Flutter companion app Auth: JWT, SSO, RBAC (Admin, Teacher, Student, Parent) Deployment: Docker, Nginx	Adaptive Learning Engine powered by Bayesian Knowledge Tracing, Emotion AI for real-time student sentiment analysis, LLM-powered AI teaching assistant for doubt resolution, AR/VR immersive lab simulations, Metaverse classroom for remote collaboration
SIH1624	Government of NCT of Delhi	Software	To develop an Artificial Intelligence (AI) based model for electricity demand projection including peak demand projection for Delhi Power system	Frontend: React.js, Plotly.js / D3.js for interactive demand curves, HTML5, CSS3, JavaScript Backend: Python (FastAPI / Django), Celery for scheduled forecasting jobs Database: PostgreSQL + TimescaleDB (historical load data), Redis AI/ML: LSTM / GRU (PyTorch/TensorFlow) for time-series load forecasting, Prophet (seasonal decomposition), XGBoost / LightGBM for feature-based prediction, Temporal Fusion Transformer for multi-horizon forecasting Features: Temperature, humidity (IMD API), calendar events, holidays, historical load curves, socioeconomic indicators Peak Detection: Signal processing (SciPy find_peaks), peak demand confidence intervals, coincidence factor analysis Visualization: Load duration curves, peak demand heatmaps, scenario comparison (normal/hot/extreme days) Export: CSV, Excel, PDF reports Deployment: Docker, Nginx, GPU for model training	Transformer-based probabilistic load forecasting with uncertainty quantification, Physics-informed constraints (power grid topology awareness), Federated Learning across DISCOMs for collaborative forecasting, Digital Twin of Delhi power grid, Reinforcement Learning for demand-side management recommendations

PS No.	Organization	Cat.	Problem Statement	Standard Tech Stack (Competition-Winning)	Advanced / Buzzword Tech
SIH1623	Government of NCT of Delhi	Software	Real-time monitoring and evaluation software for application received in Fire Department relating to inspections, follow-ups, issue of NOCs	Frontend: React.js, HTML5, CSS3, JavaScript, Material UI, Kanban board view Backend: Python (Django / FastAPI), Node.js Database: PostgreSQL (applications, inspections, NOCs), Redis (caching) Workflow Engine: Django-viewflow / custom state machine for multi-stage application tracking (Received → Assigned → Inspected → Approved/Rejected → NOC Issued) GIS: Leaflet.js / Google Maps API for building location mapping, PostGIS for spatial queries Features: Application form builder, document upload, inspector assignment and scheduling, inspection checklist (digital), photo evidence upload, auto-generated NOC certificate, follow-up task management Mobile: React Native / Flutter for field inspectors (offline capable) Notifications: SMS, email, push for status updates Analytics: Application processing time, inspector workload, bottleneck identification, Chart.js dashboards Auth: JWT, RBAC, digital signature for NOC issuance	AI-powered risk scoring for priority-based inspection scheduling, Computer Vision for fire safety compliance detection from uploaded photos, Blockchain-based tamper-proof NOC certificate verification, Digital Twin of building fire safety systems, LLM-powered automated inspection report generation
SIH1622	Government of NCT of Delhi	Software	Online issuance of Caste and other certificates by Revenue Department need real-time monitoring	Frontend: React.js, HTML5, CSS3, JavaScript, form wizard UI Backend: Python (Django / FastAPI), Node.js Database: PostgreSQL (applications, citizen data), Redis Workflow: Multi-stage approval pipeline (Apply → Verify → Approve → Issue), SLA timers with auto-escalation, Django-viewflow / custom state machine Document Verification: Tesseract OCR for uploaded document reading, Aadhaar eKYC API, Digilocker API for document fetch Monitoring: Real-time application status dashboard, processing time analytics, pending vs completed metrics, bottleneck identification by office/officer Certificate Generation: QR-coded digital certificate (cryptographic hash), ReportLab for PDF generation, digital signature (DSC/eSign API) Mobile: React Native / Flutter for citizen-facing app Notifications: SMS, email, WhatsApp Business API Auth: Aadhaar-based eKYC, JWT, RBAC	Blockchain-based verifiable credentials for tamper-proof certificates, AI-powered document fraud detection, Chatbot for application status tracking and guidance, Predictive SLA compliance scoring using historical data, W3C Verifiable Credentials standard compliance
SIH1617	Government of NCT of Delhi	Software	Dynamic route rationalization model based on machine learning/AI would be required based on real-time traffic and road parameters.	Frontend: React.js, Leaflet.js / Mapbox GL JS for route visualization, HTML5, CSS3, JavaScript Backend: Python (FastAPI / Django), Node.js Database: PostgreSQL + PostGIS (road network, stops), Redis (real-time cache), TimescaleDB (traffic time-series) Routing: OSRM / Valhalla / GraphHopper for route computation, OpenStreetMap road network data, Dijkstra / A* with dynamic edge weights AI/ML: XGBoost / LightGBM for travel time prediction, clustering (DBSCAN) for demand hotspot identification, genetic algorithm (DEAP) for multi-objective route optimization (cost, time, coverage) Traffic Data: Google Maps Directions API (live traffic), ITMS (Intelligent Traffic Management System) feeds, GTFS data for existing routes Simulation: SUMO (Simulation of Urban Mobility) for route testing Analytics: Route efficiency metrics, ridership coverage analysis, Plotly / D3.js dashboards Deployment: Docker, Nginx	Deep Reinforcement Learning for real-time adaptive routing, Digital Twin of Delhi transport network, Multi-Agent Simulation for emergent traffic patterns, Transformer-based traffic flow prediction, Quantum-inspired optimization for large-scale vehicle routing problems
SIH1607	Department of Science and Technology	Software	A smart AI based solution for traffic management on routes with heavy traffic from different directions, with real-time monitoring and adaptation of traffic light timings	Frontend: React.js, Leaflet.js / Mapbox for junction map, HTML5, CSS3, JavaScript, D3.js for traffic flow animation Backend: Python (FastAPI), Node.js Computer Vision: YOLOv8 for vehicle detection and counting from CCTV feeds, OpenCV for video stream processing, vehicle classification (car, bus, truck, two-wheeler) Traffic Signal Control: Reinforcement Learning (Deep Q-Network / PPO) for adaptive signal timing optimization, SUMO simulator for RL training environment, Webster's formula as baseline Communication: RTSP for CCTV feeds, MQTT for signal controller commands, REST API for integration with traffic management center Database: PostgreSQL + TimescaleDB (traffic counts), Redis Hardware Integration: Traffic signal controller interface (NTCIP / serial protocol), Raspberry Pi / Jetson Nano at each junction for edge inference Analytics: Congestion index, average wait time, green wave analysis Deployment: Docker, edge deployment at intersections	Multi-Agent Reinforcement Learning (MARL) for coordinated signal control, V2I (Vehicle-to-Infrastructure) communication via 5G, Graph Neural Networks for city-wide traffic flow prediction, Digital Twin of intersections for simulation-based policy testing, Federated Learning across intersections for privacy-preserving model improvement

PS No.	Organization	Cat.	Problem Statement	Standard Tech Stack (Competition-Winning)	Advanced / Buzzword Tech
SIH1604	Bharat Electronics Limited (BEL)	Software	Conversational Image Recognition Chatbot	Frontend: React.js, HTML5, CSS3, JavaScript, drag-and-drop image upload, chat interface with image preview Backend: Python (FastAPI / Flask) Vision AI: OpenAI GPT-4V / Google Gemini Vision API / open-source: LLaVA, InternVL (multimodal LLM), YOLOv8 for object detection, ResNet/EfficientNet for classification NLP: LangChain for conversation management, RAG for context-aware responses with image metadata, FAISS / ChromaDB for visual embeddings Image Processing: OpenCV, Pillow, CLIP (OpenAI) for image-text matching Database: PostgreSQL (conversations), MinIO (image storage), Redis (session cache) Voice: Whisper for speech-to-text, Web Speech API Deployment: Docker, GPU server (CUDA), Nginx	Vision-Language Foundation Model (GPT-4o / Gemini 1.5 Pro), Visual Grounding - point and ask about specific image regions, Agentic multimodal reasoning with tool-use, Real-time video stream understanding with temporal context, Neural image search across conversation history
SIH1600	AICTE, MIC-Student Innovation	Software	Student Innovation	Open-ended — Recommended Base Stack: Frontend: React.js / Next.js, HTML5, CSS3, JavaScript Backend: Python (FastAPI / Django) or Node.js (Express) Database: PostgreSQL / MongoDB, Redis AI/ML: scikit-learn, PyTorch / TensorFlow (if AI component needed) Mobile: Flutter / React Native (if mobile app required) Cloud: AWS / GCP / Azure free tier, Vercel for frontend DevOps: Docker, GitHub Actions CI/CD Tools: Figma (UI design), Git (version control), Postman (API testing)	Open-ended — choose buzzwords relevant to your specific innovation: GenAI, LLM, Blockchain, IoT, AR/VR, Digital Twin, Edge AI, Quantum Computing, Web3, Metaverse
SIH1569	Ministry of Jal Shakti	Hardware	Personalized testing kits for testing Residual Chlorine level at delivery points	Hardware: DPD colorimetric method (DPD reagent tablets/drops), portable colorimeter (LED 515nm + photodiode), ESP32/Arduino Nano MCU, OLED display (SSD1306), 3D-printed cuvette holder and enclosure, amperometric sensor alternative (bare platinum electrodes) Optics: TSL2591 lux sensor / OPT3001 for absorbance, Beer-Lambert law calibration Mobile App: Flutter / React Native, camera-based color analysis (OpenCV + color space matching), BLE for data transfer from device Backend: Python (FastAPI), PostgreSQL for test result logging, GPS tagging Compliance: IS 10500 standards comparison engine, WHO guideline thresholds Calibration: Known chlorine standard solutions, polynomial regression (NumPy) Reports: Auto-generated compliance PDF reports	IoT-enabled mesh network of chlorine sensors across water distribution, AI-based color gradient analysis using transfer learning, Blockchain-verified water quality certificates, Electrochemical impedance spectroscopy for advanced chlorine speciation, Cloud-connected predictive chlorine decay modeling
SIH1559	Ministry of Electronics and Information Technology	Hardware	Develop a functional solution that demonstrates the hardware enabled root of trust.	Hardware: TPM 2.0 module (Infineon SLB9670 / STMicroelectronics), or HSM (Hardware Security Module), secure microcontroller (STM32L5 with TrustZone / NXP LPC55S69), Raspberry Pi with TPM HAT for demo Secure Boot: U-Boot with verified boot chain, certificate-based firmware signing (OpenSSL), hash chain verification at each boot stage Software: tpm2-tools (Linux TPM interface), tpm2-ss (TPM Software Stack), Python (tpm2-pytss) for scripting TPM operations Cryptography: RSA/ECDSA key generation within TPM, sealed storage (bind data to platform state via PCR), remote attestation protocol implementation Demo Application: Secure key storage, measured boot log viewer, firmware integrity verification tool GUI: PyQt5 / Electron.js for visualization of trust chain Documentation: TCG (Trusted Computing Group) specifications reference	DICE (Device Identifier Composition Engine) for IoT device attestation, Confidential Computing with Intel SGX / AMD SEV enclaves, Blockchain-anchored device identity registry, Post-quantum cryptography integration (CRYSTALS-Kyber), Formal verification of boot chain using model checking
SIH1558	Ministry of Culture	Hardware	AI-based automated defective exhibit identification system placed in a gallery.	Hardware: Raspberry Pi 5 / NVIDIA Jetson Nano, high-resolution camera (RPi HQ Camera / USB industrial camera), motorized pan-tilt mount for automated scanning, LED lighting rig for consistent illumination, environmental sensors (DHT22 for temp/humidity, LDR for light levels) Computer Vision: OpenCV for image preprocessing, YOLOv8 / Detectron2 for defect detection (cracks, discoloration, chipping), anomaly detection (Autoencoders) for novel defects, image stitching for large exhibit panoramas ML Framework: PyTorch / TensorFlow, custom dataset: annotated exhibit defect images, transfer learning from ImageNet Backend: Python (FastAPI), PostgreSQL (defect logs, exhibit registry) Dashboard: React.js web dashboard, defect heatmap overlay on exhibit images Alerts: Email, SMS, dashboard notification on defect detection Deployment: Edge device at gallery, Docker on Jetson	Multispectral imaging for hidden damage detection (UV, IR), 3D scanning (structured light) for surface deformation analysis, Vision Transformer for fine-grained defect classification, Digital Twin of gallery with virtual exhibit condition tracking, AR-based defect annotation for museum conservators

PS No.	Organization	Cat.	Problem Statement	Standard Tech Stack (Competition-Winning)	Advanced / Buzzword Tech
SIH1557	Ministry of Coal	Software+Hardware	Development of Tyre Maintenance and Operation App, including fitment of necessary IIoT related hardware in Dumpers	Hardware: TPMS sensors (tire pressure + temperature), ESP32/STM32 MCU, CAN bus interface for OBD data, vibration sensor (ADXL345) for road condition, GPS module (NEO-6M), 4G/LoRa module for data transmission, ruggedized enclosure (IP67 for mining environment) Mobile App: Flutter / React Native (cross-platform), offline-first Backend: Python (Django / FastAPI), Node.js Database: PostgreSQL + TimescaleDB (sensor time-series), Redis IoT: MQTT broker (Mosquitto), REST API for data ingestion Analytics: scikit-learn / XGBoost for tire wear prediction, remaining useful life (RUL) estimation, TKPH (Tonne Kilometres Per Hour) calculation engine, pandas, NumPy Dashboard: React.js, Chart.js, tire pressure/temp heatmap visualization Alerts: Low pressure, overheating, predicted blowout warnings Deployment: Docker, edge server at mine site	Digital Twin of dumper tire with physics-based wear model, Predictive Maintenance 4.0 with survival analysis, Computer Vision for visual tire damage assessment, Blockchain-based tire lifecycle tracking and warranty management, Federated Learning across mine sites for improved tire life prediction
SIH1544	Autodesk	Hardware	Students are supposed to use Fusion software to generate NC code with machine details & tool library for any industrial component.	CAD/CAM: Autodesk Fusion 360 (mandatory - free educational license), SolidWorks / FreeCAD as supplementary design tools CNC Programming: Fusion 360 Manufacturing workspace, G-code / M-code generation, toolpath strategies (adaptive clearing, parallel, contour, bore, thread milling) Tool Library: Fusion 360 Tool Library (define cutter geometry, feeds, speeds), HSMAdvisor / Machinist Calculator for feeds/speeds optimization Simulation: Fusion 360 built-in toolpath simulation, collision detection, material removal verification Post Processing: Custom post processor (JavaScript-based in Fusion), machine-specific post (Fanuc, Haas, Siemens, Mazak) Materials: Aluminum, steel, titanium — material library with cutting parameters Documentation: Technical drawing (Fusion 360 Drawing workspace), setup sheets, tool lists Hardware: 3-axis / 5-axis CNC mill for physical validation (if available)	AI-powered toolpath optimization (Fusion Generative Design for manufacturing), Cloud-based multi-axis simulation, Topology-optimized fixtures via generative design, Digital Twin of CNC machine for virtual commissioning, ML-based adaptive feed rate optimization from spindle load data
SIH1543	Autodesk	Hardware	Research and redesign a conventional aerospace component commonly found in air vehicles and utilize Fusion software to reimagine its design.	CAD/CAM: Autodesk Fusion 360 (mandatory), Generative Design workspace, Topology Optimization, Shape Optimization Simulation: Fusion 360 Simulation (FEA - static stress, modal, thermal), Ansys / SimScale (cloud FEA) for advanced validation Generative Design: Define preserve/obstacle geometries, load cases, materials (Al 7075, Ti-6Al-4V, Inconel 718), manufacturing constraints (additive, CNC, casting) Additive Manufacturing: Fusion 360 Additive Build workspace, support structure generation, build orientation optimization, SLM/DMLS process parameter awareness Materials: Aerospace-grade alloys, composites, Fusion material library + custom material cards Documentation: Technical report, weight reduction analysis, stress-to-weight ratio comparison (original vs redesigned), 3D PDF export for presentation 3D Printing: FDM prototype (PLA/ABS), SLA for high-detail, metal AM (if access available)	AI-driven generative design with multi-physics constraints, Lattice structure optimization for weight reduction, Digital Thread from design to manufacturing, Multi-material topology optimization, Biomimicry-inspired structural optimization
SIH1540	AICTE, MIC-Student Innovation	Hardware	Student Innovation	Open-ended — Recommended Base Stack: Microcontroller: Arduino Uno/Mega, ESP32 (Wi-Fi+BLE), Raspberry Pi Sensors: As per project needs (DHT22, ultrasonic, IR, IMU, etc.) Communication: Wi-Fi, BLE, LoRa, 4G module as needed Firmware: C/C++ (Arduino IDE / PlatformIO), MicroPython CAD: Fusion 360 / FreeCAD for enclosure design, 3D printing Backend: Python (Flask/FastAPI), Firebase for rapid prototyping Dashboard: React.js or Blynk IoT for quick visualization PCB: KiCad / EasyEDA for custom PCB design Power: LiPo batteries, solar panel, voltage regulators	Open-ended — choose based on your project: Edge AI, TinyML, Neuromorphic, Digital Twin, Swarm Robotics, Soft Robotics, Brain-Computer Interface, Quantum Sensing

SIH 2023 (36 Statements)

PS No.	Organization	Cat.	Problem Statement	Standard Tech Stack (Competition-Winning)	Advanced / Buzzword Tech
SIH1301	Ministry of power	Software	Language translator tool to convert English to Hindi (official Language) which can be used by all the government organizations websites officially.	Frontend: React.js, HTML5, CSS3, JavaScript, WYSIWYG editor integration (TinyMCE/CKEditor with translation plugin) Backend: Python (FastAPI / Django), Node.js Translation: Bhashini API (Dhruva) for English→Hindi NMT, IndicTrans2 (AI4Bharat open-source model), fallback: Google Translate API / Azure Translator NLP: spaCy / Stanza for sentence segmentation, custom glossary for government terminology (domain-specific dictionary), post-editing interface for human review CMS Integration: WordPress plugin / Drupal module / JavaScript snippet for any website embedding Caching: Redis for translation caching (avoid re-translating same phrases) Database: PostgreSQL (translation memory, glossary) API: RESTful API with API key management for govt org onboarding Deployment: Docker, Nginx, NIC Cloud	Fine-tuned IndicTrans2 on government domain corpus, Adaptive Neural Machine Translation with user feedback loop, Transliteration-aware translation for proper nouns, Browser extension for auto-translation of any govt website, LLM-powered context-aware translation with document understanding
SIH1305	Government of Punjab	Software	The Employment Department has a digital platform www.pgrkam.com. Currently, there is no hand-holding mechanism to help the user navigate across multiple modules on the portal/app to search for answers.	Frontend: React.js / Next.js, HTML5, CSS3, JavaScript, chat widget Backend: Python (FastAPI / Django), Node.js Chatbot/Assistant: LangChain + RAG with platform content as knowledge base, intent classification (Rasa / Dialogflow / custom BERT classifier), guided navigation flows (decision tree + NLU hybrid), Bhashini API for Punjabi + Hindi + English Search: Elasticsearch for full-text search across all modules, semantic search (sentence-transformers + FAISS) for natural language queries Personalization: User profile-based recommendations (collaborative filtering), session history tracking for contextual suggestions Database: PostgreSQL (user interactions, preferences), Redis Voice: Whisper / Bhashini ASR for voice-based navigation Analytics: User journey mapping, drop-off analysis, popular query tracking Deployment: Docker, Nginx	Agentic AI Navigator with multi-step task execution (auto-fill forms, search jobs), Visual Site Map AI that understands page relationships, Proactive push notifications based on user profile matching, LLM-powered career counseling chatbot, Voice-first interface with Punjabi dialect support
SIH1312	Ministry of coal	Software	Chatbot to respond to text queries pertaining to various Acts, Rules, and Regulations applicable to Mining industries	Frontend: React.js, HTML5, CSS3, JavaScript, chat UI Backend: Python (FastAPI / Flask) NLP/AI: LangChain + RAG, FAISS / ChromaDB, sentence-transformers for embeddings, OpenAI / Gemini API / Llama 3 (Ollama) for generation, Bhashini API for Hindi support Knowledge Base: Mines Act 1952, MMDR Act 2015, Coal Mines Regulations 2017, DGMS circulars — PDF ingestion via PyMuPDF, chunking strategy Citation: Source attribution (section, rule, page) in responses Database: PostgreSQL (query logs), Redis (sessions) Testing: Ground-truth Q&A; pairs for evaluation (BLEU, ROUGE, human eval) Deployment: Docker, Nginx, Gunicorn	Legal-domain LLM fine-tuned on Indian mining legislation, Knowledge Graph of Acts → Sections → Rules → Amendments, Multi-hop reasoning for complex regulatory queries, Hallucination detection with confidence scoring, Auto-updated knowledge base when new circulars are published
SIH1314	Ministry of coal	Software	Real-time visibility of Dumper load status to Shovel operator	Frontend: React.js / custom HMI (Human-Machine Interface), HTML5, CSS3, JavaScript, real-time status indicators Backend: Python (FastAPI) / Node.js (Express) with WebSocket IoT: Load cell sensors on dumpers, ESP32/STM32 MCU, GPS modules (NEO-6M) for location, MQTT broker (Mosquitto) for real-time data streaming Communication: LoRaWAN for mine-site communication, 4G/LTE fallback, mesh networking Display: Ruggedized tablet/display in shovel cabin, color-coded dumper status (empty/loading/full/in-transit) Database: PostgreSQL + TimescaleDB, InfluxDB alternative Analytics: Cycle time analysis, payload optimization, queue time at shovel, pandas + Chart.js Alerts: Overloading warning, proximity alerts Deployment: Edge server at mine site, Docker	Digital Twin of mine fleet for simulation-based dispatch optimization, Computer Vision for payload estimation from dumper body imaging, V2V (Vehicle-to-Vehicle) mesh communication, AI-based dynamic fleet dispatch optimization, AR HUD overlay for shovel operator showing dumper status
SIH1317	Ministry of coal	Software	Development of a Digital Platform for multi-modal visibility of coal transportation	Frontend: React.js, Leaflet.js / Mapbox for multi-modal route visualization, HTML5, CSS3, JavaScript, D3.js for Sankey diagrams Backend: Python (Django / FastAPI), Node.js Database: PostgreSQL + PostGIS (routes, terminals), TimescaleDB (time-series tracking data) Data Integration: Railway tracking (FOIS API), road transport (GPS tracker APIs), port/shipping data (vessel AIS data - MarineTraffic API), belt conveyor telemetry, manual entry for rail rake placement GIS: Route mapping (road, rail, waterway), terminal/depot locations, GeoPandas, PostGIS spatial queries Analytics: Transit time analysis, mode-wise throughput, bottleneck identification, pandas, Plotly dashboards Notifications: Delay alerts, ETA updates (SMS, email, push) Auth: JWT, RBAC (Ministry, Company, Transporter, Terminal) Deployment: Docker, Nginx, message queue (RabbitMQ/Kafka) for high-throughput data	Supply Chain Digital Twin for end-to-end coal logistics simulation, AI-based ETA prediction using multi-modal transit data, Blockchain-based consignment tracking for tamper-proof coal movement records, Satellite imagery for coal stockpile volume estimation, Graph-based optimal mode selection engine

PS No.	Organization	Cat.	Problem Statement	Standard Tech Stack (Competition-Winning)	Advanced / Buzzword Tech
SIH1319	Ministry of coal	Software	Forecasting and scheduling of railway rakes.	Frontend: React.js, FullCalendar.js / Gantt chart for schedule visualization, HTML5, CSS3, JavaScript Backend: Python (Django / FastAPI) Database: PostgreSQL (rake inventory, route data, demand), TimescaleDB (historical rake movement time-series) Forecasting: ARIMA / SARIMA, Prophet for seasonal demand, LSTM (PyTorch/TensorFlow) for complex temporal patterns, XGBoost for feature-based forecasting (production schedule, stock levels, weather) Scheduling: Google OR-Tools (job shop scheduling, constraint programming), PuLP for LP/ILP, genetic algorithm (DEAP) for multi-objective rake allocation Data: FOIS (Freight Operations Information System) integration, coal production forecasts, port/siding capacity constraints Reporting: Rake utilization metrics, turnaround time analysis, Chart.js / Plotly dashboards, PDF reports Deployment: Docker, Nginx	Deep Reinforcement Learning for dynamic rake rescheduling, Digital Twin of rail freight network, Quantum-inspired optimization for NP-hard scheduling, Multi-Agent System for decentralized rake coordination, Real-time disruption management using event-driven AI
SIH1321	Ministry of Chemicals and Fertilizers	Software	Compilation and Collection of optimum Level (variable) of data set from the Chemical and Petrochemical Industries in the country.	Frontend: React.js / Angular, HTML5, CSS3, JavaScript, data table with advanced filters (AG Grid), form builder Backend: Python (Django / FastAPI), Node.js Database: PostgreSQL (structured industry data), Elasticsearch (search and discovery), MongoDB (semi-structured submissions) Data Collection: Dynamic form builder (react-jsonschema-form), Excel/CSV bulk upload (pandas + openpyxl), API-based data submission for automated systems Data Quality: pandas-profiling (ydata-profiling) for automated data quality reports, Great Expectations for data validation rules, fuzzy deduplication (RapidFuzz) Analytics: pandas, NumPy, Plotly / D3.js for industry dashboards, trend analysis, state-wise comparison Reporting: ReportLab, openpyxl for Excel exports, scheduled report generation (Celery) Auth: JWT, RBAC (Ministry, Industry, Analyst), API key management Deployment: Docker, Nginx	AI-powered data imputation for missing values, NLP-based automated variable identification from unstructured reports, Knowledge Graph of chemical industry value chains, Federated data collection preserving industry data privacy, LLM-powered natural language analytics query interface
SIH1329	Ministry of Information and Broadcasting	Software	360-degree feedback software for the Government of India related News Stories in Regional Media using AI/ML	Frontend: React.js / Next.js, HTML5, CSS3, JavaScript, D3.js for analytics Backend: Python (Django / FastAPI) Data Collection: Web scraping (Scrapy / BeautifulSoup) for regional news portals, RSS feed aggregation, social media APIs (Twitter/X, Facebook Graph API), YouTube Data API for video news NLP: Multilingual NER (spaCy + Stanza) for entity extraction, sentiment analysis (IndicBERT / multilingual-BERT), topic modeling (LDA / BERTopic), text summarization (mT5 / IndicBART) Translation: Bhashini API / IndicTrans2 for 13 regional languages → English Database: PostgreSQL (articles, sentiment scores), Elasticsearch (search) Analytics: Sentiment trend over time, topic clusters, regional coverage heatmap, media outlet bias scoring Reporting: Auto-generated daily/weekly media perception reports (ReportLab) Deployment: Docker, Nginx, Celery for scheduled crawling	LLM-powered media narrative analysis and framing detection, Misinformation/fake news detection using claim verification models, Real-time media monitoring with streaming NLP pipeline (Apache Kafka), Multimodal analysis (image + text) of news stories, AI-generated counter-narrative suggestions
SIH1330	Ministry of Information and Broadcasting	Software	Text to Video of various PIB Press Releases using AI/ML/GANs in English and 13 Regional Languages.	Frontend: React.js, HTML5, CSS3, JavaScript, video player (Video.js) Backend: Python (FastAPI / Django), Celery for async video generation Text Processing: NLP pipeline: summarization (T5/BART), key point extraction, script generation from press release Translation: Bhashini API / IndicTrans2 for 13 regional languages Text-to-Speech: Bhashini TTS / Google TTS / Coqui TTS for multilingual voice, SSML for intonation control Video Generation: FFmpeg for video composition, MoviePy for programmatic video editing, Remotion (React-based video) for template-based generation, stock footage/image overlay with Ken Burns effect Avatars: D-ID / SadTalker for talking-head avatar from text Storage: MinIO / S3 for video storage, CDN for delivery Database: PostgreSQL (press releases, generated videos) Deployment: Docker, GPU server for avatar/video generation	Diffusion-based video generation (Sora / Runway Gen-3), Neural Avatar with real-time lip sync and emotion, Generative Adversarial Networks for photorealistic scene generation, Voice cloning for consistent narrator across languages, Real-time text-to-video pipeline with streaming output

PS No.	Organization	Cat.	Problem Statement	Standard Tech Stack (Competition-Winning)	Advanced / Buzzword Tech
SIH1333	Ministry of Social Justice and Empowerment	Software	Udyog Saarthi App - (Progressive Web-based Application) for Adults undergoing Job coaching for opportunities under 4% reservation in NIEPMD and other Institutions.	Frontend: React.js (PWA - service workers, manifest.json), HTML5, CSS3, JavaScript, responsive design, accessibility (WCAG 2.1 AA) Backend: Python (Django / FastAPI), Node.js Database: PostgreSQL (user profiles, job listings, coaching logs), Redis Features: Job search and matching, coaching session scheduling, skill assessment quizzes, progress tracking, resume builder, employer portal for posting opportunities Accessibility: Screen reader support (ARIA labels), high contrast mode, text-to-speech integration, adjustable font sizes, keyboard navigation, sign language video support Matching: TF-IDF / sentence-transformers for skill-job matching, collaborative filtering for job recommendations Notifications: Push (service worker), SMS, email Offline: PWA with offline caching (Workbox) Auth: JWT, Aadhaar-based verification option Deployment: Vercel / Netlify (frontend), Docker (backend)	AI-powered personalized career path recommendation, NLP-based resume optimization for job descriptions, VR-based job simulation for experiential coaching, LLM chatbot for 24/7 career guidance, Predictive placement scoring using ML
SIH1348	Ministry of Railways	Software	Natural language translation engine for announcements and information dissemination at stations	Frontend: React.js, HTML5, CSS3, JavaScript, admin panel for announcement management Backend: Python (FastAPI / Django) Translation: Bhashini API (Dhruva) for real-time NMT, IndicTrans2 (AI4Bharat) for offline/self-hosted translation, language detection (langdetect / fastText lid model) Text-to-Speech: Bhashini TTS / Google TTS / Coqui TTS for 22+ Indian languages, SSML for pronunciation and pace control Speech-to-Text: Whisper / Bhashini ASR for voice-based announcement input Template Engine: Pre-built announcement templates (train arrival/departure, delay, platform change) with variable slots, auto-translation of templates Output: Audio playback system (PA system integration via audio output), LED/LCD display text feed, mobile app notification Database: PostgreSQL (templates, translation memory), Redis (cache) Deployment: Docker, edge server at station, Nginx	Zero-shot cross-lingual transfer for unseen language pairs, Neural voice cloning for station-specific announcer voice, Streaming NMT for real-time simultaneous translation, Emotion-appropriate TTS for emergency vs routine announcements, Lip-sync video avatar for visual announcement displays
SIH1349	Ministry of Railways	Software	Using existing CCTV network for crowd management, crime prevention, and work monitoring using AI/ML	Frontend: React.js, HTML5, CSS3, JavaScript, video grid dashboard Backend: Python (FastAPI / Django), Node.js Video Processing: OpenCV, FFmpeg, GStreamer, RTSP stream handling Computer Vision: YOLOv8 for person/object detection, DeepSORT / ByteTrack for multi-object tracking, crowd density estimation (CSRNet / MCNN), pose estimation (MediaPipe / OpenPose) for suspicious activity detection Crime Prevention: Anomaly detection (autoencoders on normal behavior patterns), abandoned object detection, loitering detection (dwell time thresholds), face recognition (ArcFace / FaceNet) for watchlist matching Work Monitoring: Zone-based personnel presence tracking, activity classification (cleaning, maintenance, idle) Database: PostgreSQL, TimescaleDB (event logs), MinIO (video clips) Alerts: Real-time notifications (WebSocket), SMS, siren trigger Hardware: NVIDIA Jetson Orin / GPU server for edge inference Deployment: Docker, Kubernetes for scale, CUDA	Vision Transformer for action recognition, Graph Neural Network for social group behavior analysis, Federated Learning across stations for privacy-preserving model improvement, Generative AI for synthetic training data (rare crime scenarios), Explainable AI for evidence-grade incident reporting
SIH1350	Government of Jammu and Kashmir	Software	Artificial Intelligence Driven Digitization of Cadastral Maps	Frontend: React.js, Leaflet.js / OpenLayers for map display, HTML5, CSS3, JavaScript Backend: Python (Django / FastAPI) Image Processing: OpenCV (binarization, deskewing, noise removal), Pillow for image manipulation, high-resolution scanning (600+ DPI recommended) AI/ML: U-Net / Mask R-CNN for parcel boundary segmentation, Tesseract / PaddleOCR for text extraction from maps (parcel numbers, names), YOLOv8 for symbol detection (landmarks, markers) GIS: GDAL for georeferencing (affine transformation, GCP-based rubber sheeting), PostGIS for vector parcel storage, GeoJSON / Shapefile export, coordinate transformation (pyproj) Validation: Topology checks (no gaps/overlaps), area comparison with revenue records Database: PostgreSQL + PostGIS Deployment: Docker, GPU server for segmentation inference	Foundation Models (SAM - Segment Anything) for zero-shot parcel boundary detection, GAN-based image enhancement for degraded old maps, Graph Neural Networks for parcel topology inference, Blockchain-based land record immutability, 3D cadastre using LiDAR point cloud integration

PS No.	Organization	Cat.	Problem Statement	Standard Tech Stack (Competition-Winning)	Advanced / Buzzword Tech
SIH1351	Government of Jammu and Kashmir	Software	Digital Generator Monitoring - Diesel generators are widely used as backup power sources in industries, homes, and remote locations	IoT Hardware: ESP32 / Arduino Mega MCU, current sensor (ACS712/SCT-013), voltage sensor (ZMPT101B), fuel level sensor (ultrasonic / float), RPM sensor (hall effect), temperature sensor (DS18B20 / thermocouple), vibration sensor (ADXL345), GPS module Communication: MQTT (Mosquitto broker), Wi-Fi/4G module Frontend: React.js, Chart.js / Grafana for real-time dashboards Backend: Python (FastAPI / Django), Node.js Database: PostgreSQL + TimescaleDB / InfluxDB Analytics: Fuel consumption rate calculation, efficiency metrics, runtime tracking, maintenance scheduling, anomaly detection (scikit-learn - Isolation Forest) Alerts: Overheating, low fuel, overload, maintenance due (SMS, email, push) Mobile: Flutter / React Native companion app Deployment: Edge server + cloud sync, Docker	Predictive Maintenance using remaining useful life (RUL) estimation, Digital Twin of generator for simulation and optimization, AI-based fuel theft detection from consumption anomalies, Federated Learning across generator fleet for improved models, Carbon footprint tracking and emission estimation
SIH1359	Ministry of Commerce and Industries	Software	Contact center knowledge management tool with decision tree	Frontend: React.js / Angular, HTML5, CSS3, JavaScript, decision tree visualizer (D3.js / react-decision-tree), rich text editor for knowledge articles Backend: Python (Django / FastAPI), Node.js Database: PostgreSQL (knowledge articles, decision trees), Elasticsearch (full-text search across articles) Decision Tree Builder: Drag-and-drop tree builder UI (react-flow / GoJS), conditional logic engine (if-then-else with nested conditions), multi-path branching with endpoint resolutions Knowledge Base: Article CRUD with versioning, categorization and tagging, FAQ management, media embedding (images, PDFs) Search: Elasticsearch full-text + semantic search (sentence-transformers + FAISS) Analytics: Article usage metrics, decision tree path analysis, agent performance tracking, Chart.js dashboards Auth: JWT, RBAC (Admin, Supervisor, Agent) Deployment: Docker, Nginx	LLM-powered auto-answer suggestion from knowledge base, AI-generated decision trees from historical ticket resolution data, Conversational AI copilot for contact center agents, Sentiment analysis for call quality monitoring, Auto-updated knowledge base using NLP on resolved tickets
SIH1360	Ministry of Commerce and Industries	Software	Price comparison of GeM products with other e-marketplaces	Frontend: React.js, HTML5, CSS3, JavaScript, comparison table UI, price chart (Chart.js / Plotly) Backend: Python (Django / FastAPI), Node.js Web Scraping: Scrapy / Playwright / Selenium for Amazon, Flipkart, IndiaMart, GeM API (if available) for product catalog, rotating proxies, CAPTCHA handling Product Matching: TF-IDF + cosine similarity for product title matching, sentence-transformers for semantic matching, fuzzy matching (RapidFuzz) for brand/model normalization Data Pipeline: Apache Airflow / Celery for scheduled scraping jobs, pandas for data cleaning and normalization Database: PostgreSQL (product catalog, price history), Elasticsearch (product search) Analytics: Price trend analysis, average markup calculation, best-value ranking, category-wise comparison Reporting: Auto-generated comparison reports (ReportLab), Excel exports Deployment: Docker, Nginx, Celery workers	LLM-powered product attribute extraction from unstructured listings, CLIP-based visual product matching (image similarity), Real-time price monitoring with webhook alerts, AI-powered negotiation recommendation engine, Blockchain-based transparent price benchmarking
SIH1370	Government of Jharkhand	Software	Real time monitoring of infrastructure development	Frontend: React.js, HTML5, CSS3, JavaScript, Leaflet.js / Mapbox for project location map, Gantt chart (dhtmlxGantt) for timeline view Backend: Python (Django / FastAPI), Node.js Database: PostgreSQL + PostGIS (project locations), Redis Features: Project registration, milestone tracking, fund utilization monitoring, contractor management, geo-tagged photo/video progress upload, work order and bill management Mobile: Flutter / React Native for field engineers (offline + GPS auto-tag) Computer Vision: Image-based progress estimation (optional - CNN classifier for construction stage), before/after image comparison Analytics: Project health dashboard (SPI, CPI, delay analysis), district-wise progress heatmap, Chart.js / Plotly Reporting: Auto-generated progress reports (ReportLab), Excel exports Notifications: Deadline reminders, delay alerts (SMS, email, push) Auth: JWT, RBAC (State, District, Block, Contractor) Deployment: Docker, Nginx	Drone/satellite imagery-based progress verification, AI-powered delay prediction and risk scoring, Blockchain-based fund flow tracking, Digital Twin of infrastructure project, BIM (Building Information Modeling) integration for planned vs actual

PS No.	Organization	Cat.	Problem Statement	Standard Tech Stack (Competition-Winning)	Advanced / Buzzword Tech
SIH1377	Ministry of Power	Software	Creation of Live Digital Twins for power Projects and integration with all existing monitoring and database system for holistic real-time approach to construction, operation and maintenance.	Frontend: React.js, Three.js / Cesium.js for 3D visualization, D3.js / Plotly for real-time charts, HTML5, CSS3, JavaScript Backend: Python (FastAPI / Django), Node.js, WebSocket for real-time updates 3D/BIM: IFC.js for BIM model rendering in browser, Revit/AutoCAD export to IFC format, Three.js for interactive 3D plant visualization IoT Integration: MQTT broker (Mosquitto), OPC-UA for SCADA/DCS data, Modbus TCP for legacy equipment Database: PostgreSQL (project data), TimescaleDB / InfluxDB (time-series sensor data), Redis (real-time cache) Data Integration: Apache Kafka for streaming data pipeline, ETL from existing monitoring systems (PI Historian, SCADA) Analytics: Predictive maintenance (scikit-learn, XGBoost), anomaly detection on sensor streams, KPI dashboards (availability, heat rate, PLF) Simulation: MATLAB/Simulink for plant process simulation, Python (SciPy) for simplified thermal models Deployment: Docker, Kubernetes, GPU for 3D rendering	Physics-Informed Neural Networks for real-time process prediction, Extended Reality (XR) with HoloLens for immersive plant inspection, Autonomous AI agents for self-healing plant operations, Blockchain-based equipment lifecycle tracking, Generative AI for automated maintenance procedure generation
SIH1391	Ministry of Environment	Software	Behavioural change monitoring software	Frontend: React.js / Next.js (PWA), HTML5, CSS3, JavaScript, gamification UI (progress bars, badges, streaks) Backend: Python (Django / FastAPI), Node.js Database: PostgreSQL (user activities, surveys), MongoDB (flexible behavior logs), Redis Features: Behavior tracking forms (daily habits, consumption patterns), survey builder (react-jsonschema-form), pledge/commitment system, community challenges, reward points system Analytics: Behavior change metrics (pre/post intervention), cohort analysis, trend visualization (Chart.js / Plotly), statistical significance testing (SciPy) AI/ML: Clustering (k-means) for behavioral segments, recommendation engine for personalized eco-tips, sentiment analysis on user feedback Mobile: PWA or React Native for daily logging Notifications: Push reminders, streak maintenance alerts Auth: JWT, social login, anonymous mode option Deployment: Docker, Nginx	Nudge AI for behavioral economics-based intervention optimization, LLM-powered personalized coaching messages, Social Network Analysis for behavior diffusion modeling, Digital Twin of community environmental footprint, Gamified carbon credit system with token rewards
SIH1392	Ministry of Environment	Software	E-Waste Facility Locator	Frontend: React.js, Leaflet.js / Google Maps API for facility map, HTML5, CSS3, JavaScript, responsive mobile-first design Backend: Python (Django / FastAPI), Node.js Database: PostgreSQL + PostGIS (facility locations, service areas) GIS: PostGIS spatial queries (nearest facility), driving distance/time (OSRM / Google Directions API), service area polygons (isochrone maps) Data: CPCB authorized recycler list, state pollution board data, user-contributed facility additions with admin verification Features: Search by location/pincode, filter by e-waste category, pickup request booking, facility details (contact, accepted items, hours), e-waste category guide (what goes where) Mobile: PWA or React Native Notifications: Pickup confirmation, collection drive alerts Auth: JWT (optional - for pickup booking), guest access for search Deployment: Docker, Nginx, Vercel for frontend	AI-based e-waste volume estimation from user-uploaded photos, Route optimization for collection vehicle dispatch, Blockchain-based e-waste recycling certificate verification, Gamified recycling rewards with carbon credit integration, Chatbot for e-waste disposal guidance
SIH1393	Ministry of Environment	Software	Water and Electricity Tracking App	Frontend: React.js (PWA) / React Native, HTML5, CSS3, JavaScript Backend: Python (Django / FastAPI), Node.js Database: PostgreSQL (user consumption data, billing), Redis Features: Manual entry of meter readings (water + electricity), photo OCR of meter reading (Tesseract / PaddleOCR), bill upload and parsing, consumption trend charts (Chart.js), daily/weekly/monthly comparison, budget alerts, tips for reduction Analytics: Per-unit cost calculation, anomaly detection (unusual spikes), peer comparison (similar household size/area), carbon footprint estimation from electricity consumption Gamification: Conservation challenges, badges, community leaderboard IoT (optional): Smart meter integration via API, MQTT for real-time consumption data Notifications: Budget alerts, unusual consumption warnings (push, SMS) Auth: JWT, Firebase Auth Deployment: Docker, Nginx, Play Store / PWA	AI-powered consumption forecasting and bill prediction, Smart meter IoT integration for real-time tracking, Blockchain-based verifiable consumption reduction credits, LLM-powered energy saving recommendations, Digital Twin of household utility consumption

PS No.	Organization	Cat.	Problem Statement	Standard Tech Stack (Competition-Winning)	Advanced / Buzzword Tech
SIH1407	Ministry of Home Affairs	Software	Chatbot based helpdesk for govt employees and departments	Frontend: React.js, HTML5, CSS3, JavaScript, embeddable chat widget Backend: Python (FastAPI / Flask), Node.js NLP/AI: LangChain + RAG (FAISS/ChromaDB), intent classification (BERT / custom classifier), entity extraction (spaCy), fallback to human agent routing Knowledge Base: HR policies, leave rules, IT support FAQs, department-specific SOPs, form downloads, document ingestion pipeline (PDF, DOCX) Ticketing: Integrated ticketing system for unresolved queries, ticket status tracking, SLA management Database: PostgreSQL (tickets, conversations), Redis (sessions), vector DB (ChromaDB/FAISS) Analytics: Query analytics, resolution rate, popular topics, agent workload distribution Auth: LDAP/AD integration for govt network SSO, JWT Deployment: Docker, NIC Cloud, Nginx	Agentic RAG with tool-use (auto-check leave balance, file IT ticket), Voice-enabled helpdesk with Whisper + TTS, Multi-lingual support with Bhashini, Self-learning chatbot with continuous fine-tuning on resolved tickets, Emotion-aware response escalation
SIH1410	Ministry of Home Affairs	Software	Augmented/ Virtual reality system for the live training of troops	Engine: Unity 3D (C#) or Unreal Engine (C++) — primary development platform AR: ARCore (Android) / ARKit (iOS), Vuforia for marker-based AR, Microsoft Mixed Reality Toolkit (MRTK) for HoloLens VR: Oculus SDK / SteamVR / OpenXR for cross-platform VR, hand tracking (Oculus Quest hand tracking / Ultraleap) 3D Modeling: Blender for environment and equipment models, terrain generation (World Machine / Gaia), military equipment 3D assets Simulation: Physics engine (Unity Physics / PhysX), AI-driven enemy behavior (behavior trees / NavMesh AI), ballistics simulation, terrain navigation Networking: Photon PUN / Mirror (Unity multiplayer) for multi-user training Backend: Node.js / Python for session management, analytics Database: PostgreSQL (training records, scores), Firebase Hardware: Meta Quest 3 / HTC Vive Pro 2, HoloLens 2 for AR scenarios Analytics: Performance scoring, decision-making assessment, after-action review	Procedural Content Generation for infinite scenario variation, AI-driven adaptive difficulty based on trainee performance, Haptic feedback integration for realistic weapon handling, Digital Twin of real terrain from satellite/drone imagery, Brain-Computer Interface for cognitive load monitoring during training
SIH1415	Ministry of Defence	Software	Development of motion amplification video techniques for vibration analysis	Video Processing: OpenCV, FFmpeg for video I/O, Python (NumPy, SciPy) for signal processing Motion Amplification: Eulerian Video Magnification (EVM) algorithm implementation, phase-based motion amplification (complex steerable pyramid), Riesz pyramid for real-time processing Signal Analysis: FFT for frequency extraction from amplified motion, spectrogram generation, modal analysis (frequency, damping ratio), SciPy signal processing (butter, filtfilt for bandpass filtering) AI/ML: PyTorch (optional) - deep learning-based motion magnification (Learning-based Video Motion Magnification), anomaly detection on extracted vibration signatures GUI: PyQt5 / Streamlit for interactive analysis tool, ROI selection, frequency range selection, amplification factor control Output: Amplified video export, vibration spectrum plots (Matplotlib), frequency report generation Camera: High-speed camera recommended (120+ fps), standard camera for low-frequency vibrations Deployment: Desktop application, Docker for server version	Physics-Informed Neural Network for vibration mode shape estimation, Real-time motion amplification on edge devices (Jetson), Digital Twin integration for vibration-based structural health monitoring, Transformer-based temporal super-resolution for video magnification, AR overlay of amplified vibration on real structure
SIH1428	Ministry of Rural Development	Software	AR/VR based application which helps visualize the complete house virtually at an early stage, which can help in cost estimation, planning and completion of the house in time	Engine: Unity 3D (C#) with AR Foundation (ARCore + ARKit), A-Frame / Three.js for web-based lightweight AR 3D Modeling: Blender / SketchUp for house models, parametric house generation from PMAY-G approved designs, modular room/wall/roof components AR: ARCore (Android) ground plane detection, place 3D house model at real-world scale, walk-through capability, room-by-room visualization Cost Estimation: BOM (Bill of Materials) calculator linked to 3D model, SOR (Schedule of Rates) database integration, region-wise material cost database, auto-generated cost breakdown report Backend: Python (Django / FastAPI), Node.js Database: PostgreSQL (house designs, material costs, project tracking) Planning: Construction phase timeline (Gantt chart), material procurement schedule, milestone tracking Mobile: React Native / Flutter with Unity embedding for AR, or standalone Unity Android/iOS app Deployment: Play Store, Docker for backend	BIM (Building Information Modeling) integration for precise quantity take-off, AI-powered construction progress prediction, Photogrammetry for as-built comparison, Generative AI for house design variation from constraints, Metaverse walkthrough for stakeholder review

PS No.	Organization	Cat.	Problem Statement	Standard Tech Stack (Competition-Winning)	Advanced / Buzzword Tech
SIH1429	Ministry of Rural Development	Software	Develop a solution to identify discrepancies in Quality Monitoring Data on OMMAS and generate reports that highlight areas where corrective action is Required.	Frontend: React.js, HTML5, CSS3, JavaScript, data tables with conditional formatting Backend: Python (Django / FastAPI) Database: PostgreSQL (OMMAS data mirror, discrepancy logs) Data Ingestion: OMMAS API / database connector, CSV/Excel upload for offline data, pandas for data loading and cleaning Discrepancy Detection: Rule-based checks (missing fields, range violations, date inconsistencies, duplicate entries), statistical outlier detection (Z-score, IQR, Grubbs test), cross-validation between related data fields, scikit-learn (Isolation Forest) for anomaly detection Reporting: Auto-generated discrepancy reports (ReportLab / WeasyPrint), state/district/block level aggregation, severity classification (Critical/Major/Minor), corrective action recommendations Dashboard: Chart.js / Plotly for trend visualization, discrepancy heatmap by geography Auth: JWT, RBAC Deployment: Docker, Nginx, scheduled batch analysis (Celery/cron)	AI-powered root cause analysis for systematic data quality issues, NLP-based automated corrective action suggestion generation, Predictive data quality scoring using historical patterns, Blockchain-based data integrity verification chain, AutoML for adaptive anomaly detection thresholds
SIH1443	Autodesk Education Experience	Software	Students to use Autodesk Fusion 360 to research and to generate NC code with machine details and tool library for any industrial component.	CAD/CAM: Autodesk Fusion 360 (mandatory), Manufacturing workspace for CAM operations CNC Programming: G-code generation via Fusion 360 toolpaths, 2D (profile, pocket, drill) and 3D (adaptive, parallel, contour) strategies, tool library setup (end mills, drills, taps, inserts) Post Processing: Fusion 360 post processor (JavaScript), machine-specific posts (Fanuc, Haas, Siemens controllers) Simulation: Fusion 360 toolpath simulation (stock simulation), collision detection, tool engagement analysis Material Library: Cutting data for aluminum, steel, stainless, plastics, feed/speed calculations (SFM, IPT, chip load) Documentation: Setup sheet generation, tool list, technical drawings (Fusion Drawing workspace) Validation: Physical machining on CNC mill if available, verify G-code with NC viewer (NCViewer.com or CAMotics)	AI-assisted toolpath optimization for minimum cycle time, Cloud machining simulation with multi-axis kinematics, Generative Design for fixture and workholding, IoT-connected CNC for real-time machining parameter feedback, Digital Thread from CAD to quality inspection
SIH1449	National Technical Research Organisation, (NTRO)	Software	Design, develop and implement a software bill of materials (SBOM) generation tool that can generate the complete SBOM of custom-developed software.	Core: Python (primary), Node.js (for JS ecosystem scanning) SBOM Standards: SPDX 2.3 / CycloneDX 1.5 format output (JSON, XML), NTIA minimum elements compliance Dependency Scanning: pip (requirements.txt, Pipfile, pyproject.toml), npm/yarn (package.json, package-lock.json), Maven (pom.xml), Gradle (build.gradle), Go (go.mod), Rust (Cargo.toml), C/C++ (CMakeLists.txt, conanfile), Docker (Dockerfile layer analysis) Source Analysis: Tree-sitter / AST parsing for import statement extraction, license detection (Scancode-toolkit), vulnerability cross-referencing (NVD / OSV database) GUI: React.js web dashboard for SBOM visualization (dependency tree graph - D3.js), vulnerability highlighting, license compliance checker CLI: Click / argparse for command-line usage, CI/CD integration (GitHub Actions / Jenkins plugin) Database: SQLite / PostgreSQL for SBOM history, diff tracking Deployment: pip installable package, Docker image, CLI binary	AI-powered transitive dependency risk scoring, Supply chain attack detection using anomaly analysis, Blockchain-based SBOM integrity and provenance, Binary SCA (Software Composition Analysis) for compiled artifacts, ML-based license compatibility prediction
SIH1450	National Technical Research Organisation, (NTRO)	Software	Develop and deploy a Large Language Model (LLM) based tool for generating human like responses to natural language inputs for network not connected over internet	LLM: Open-source models: Llama 3 (8B/70B), Mistral 7B, Phi-3, Gemma, deployed locally via Ollama / llama.cpp / vLLM Inference: llama.cpp (C++ optimized CPU inference), GGUF quantized models (Q4_K_M / Q5_K_M for memory efficiency), ONNX Runtime for GPU inference, vLLM for batched serving RAG (offline): LangChain + FAISS (offline vector store), sentence-transformers (all-MiniLM-L6-v2) for local embeddings, document ingestion: PyMuPDF, python-docx Frontend: React.js / Streamlit / Gradio for chat interface Backend: Python (FastAPI) Database: SQLite (conversation logs), FAISS (vector store) — all local Hardware: Minimum: 16GB RAM CPU-only (7B Q4 model), Recommended: NVIDIA GPU (RTX 3090/4090) with 24GB VRAM for 70B models Deployment: Air-gapped deployment, Docker (pre-built image with model weights), USB-transferable package	Speculative decoding for faster inference on resource-constrained hardware, LoRA/QLoRA fine-tuning on domain-specific data, Mixture-of-Experts (MoE) architecture for efficient large models, Knowledge distillation to create smaller specialized models, On-device Retrieval-Augmented Generation with local knowledge graph

PS No.	Organization	Cat.	Problem Statement	Standard Tech Stack (Competition-Winning)	Advanced / Buzzword Tech
SIH1457	MathWorks India Private Limited	Software	Get Fit Faster with a High-Tech Tracker: Analyze Human Activity and GPS Data for Improved Results	Platform: MATLAB (mandatory - MathWorks problem), Simulink Data Sources: Smartphone IMU (accelerometer, gyroscope), GPS, MATLAB Mobile app for sensor data collection Signal Processing: MATLAB Signal Processing Toolbox (FFT, filtering, windowing), Sensor Fusion and Tracking Toolbox Activity Recognition: MATLAB Classification Learner App, feature extraction (mean, std, RMS, frequency-domain features), classifiers: SVM, k-NN, Random Forest, LSTM via Deep Learning Toolbox GPS Analysis: MATLAB Mapping Toolbox for route visualization, speed/elevation profiling, distance calculation (Haversine), Kalman filter for GPS noise smoothing Fitness Metrics: Step counting (peak detection), calorie estimation, pace calculation, heart rate zone analysis (if wearable data available) Visualization: MATLAB plots, App Designer for GUI Deployment: MATLAB App Designer standalone, MATLAB Compiler for executable, ThingSpeak for cloud IoT	Deep Learning-based HAR (Human Activity Recognition) with Transformer models, Federated Learning across users for personalized fitness models, Digital Twin of human body for biomechanical analysis, Reinforcement Learning for adaptive training plan optimization, Wearable Edge AI for real-time activity classification
SIH1460	AICTE	Software	Integration of AI for Adaptive Learning for MCQ Selection in PARAKH	Frontend: React.js / Next.js, HTML5, CSS3, JavaScript Backend: Python (Django / FastAPI) Database: PostgreSQL (question bank, student responses, item parameters), Redis Item Response Theory (IRT): Python (pyirt / catsim) for 1PL/2PL/3PL IRT model estimation, item difficulty, discrimination, and guessing parameter calibration Adaptive Testing: Computerized Adaptive Testing (CAT) algorithm, maximum information criterion for next question selection, ability estimation (EAP / MLE) AI/ML: NLP for question quality analysis (readability, cognitive level classification - Bloom's taxonomy), auto-detection of flawed distractors (answer option analysis), scikit-learn for difficulty prediction from question text features Question Bank: Tagged by subject, topic, difficulty, cognitive level, version control for question edits Reporting: Item analysis reports, test reliability metrics (Cronbach's alpha), student ability distribution charts Auth: JWT, RBAC (Admin, Item Writer, Reviewer, Student) Deployment: Docker, Nginx	LLM-powered automatic MCQ generation from textbook content, Knowledge Tracing using Deep Learning (DKT), Transformer-based difficulty prediction from question semantics, Fairness-aware item selection (DIF - Differential Item Functioning analysis), Gamified adaptive assessment with engagement optimization
SIH1495	AICTE, MIC-Student Innovation	Software	Student Innovation	Open-ended — Same as SIH1600 (see above): Frontend: React.js / Next.js, HTML5, CSS3, JavaScript Backend: Python (FastAPI / Django) or Node.js (Express) Database: PostgreSQL / MongoDB, Redis AI/ML: scikit-learn, PyTorch / TensorFlow (if AI component) Mobile: Flutter / React Native Cloud: AWS / GCP / Azure free tier DevOps: Docker, Git, GitHub Actions	Open-ended — choose based on your specific innovation
SIH1498	AICTE, MIC	Software	Develop a Proctored exam tool for shortlisting the candidates for the national and international level hackathons.	Frontend: React.js, HTML5, CSS3, JavaScript, WebRTC for camera/screen capture Backend: Python (Django / FastAPI), Node.js (Express), WebSocket Database: PostgreSQL (exam data, candidate profiles), Redis (session management) Proctoring: Face detection (MediaPipe / face-api.js) for presence verification, head pose estimation for gaze tracking, multiple face detection (alert on 2+ faces), tab switch / window blur detection (Page Visibility API), screen recording (MediaRecorder API), audio monitoring for voice detection Anti-Cheating: Browser lockdown mode (full-screen enforcement), clipboard blocking, right-click disable, AI-based suspicious behavior scoring (scikit-learn classifier) Exam Engine: MCQ, coding (integrate Judge0 for code execution), timer management, auto-submit, randomized question order Code Editor: Monaco Editor (VS Code in browser), multi-language support (Python, C++, Java, JavaScript) Reporting: Candidate score cards, proctoring incident reports, anomaly flagging for manual review Auth: JWT, Aadhaar/photo ID verification, OTP Deployment: Docker, Nginx, WebSocket server, CDN	AI-powered eye tracking using gaze estimation neural networks, Keystroke dynamics analysis for identity verification, LLM-based code plagiarism detection (semantic similarity), Deepfake detection for video proctoring integrity, Blockchain-based exam certificate and score immutability

PS No.	Organization	Cat.	Problem Statement	Standard Tech Stack (Competition-Winning)	Advanced / Buzzword Tech
SIH1506	Ministry of Ports, Shipping and Waterways	Software	Leveraging the power of deep learning to overcome the challenges of marine engineering and improve vessel operations	Frontend: React.js, Leaflet.js / Mapbox for vessel tracking, HTML5, CSS3, JavaScript, D3.js for analytics Backend: Python (FastAPI / Django) Data Sources: AIS (Automatic Identification System) data (pyais), vessel performance logs (speed, fuel, RPM, draft), weather/ocean data (OpenWeatherMap API, Copernicus Marine Service) Deep Learning: PyTorch / TensorFlow, LSTM/GRU for fuel consumption prediction, CNN for engine fault pattern recognition from sensor data, time-series forecasting for ETA prediction, XGBoost / LightGBM for feature-based models Applications: Route optimization (A* with weather constraints), predictive maintenance (remaining useful life estimation), hull fouling detection from speed-power analysis, port congestion prediction Database: PostgreSQL + PostGIS, TimescaleDB Visualization: Plotly / D3.js for vessel performance dashboards Deployment: Docker, GPU server for model inference	Physics-Informed Neural Networks for hydrodynamic performance modeling, Digital Twin of vessel for real-time operational optimization, Reinforcement Learning for autonomous voyage planning, Satellite imagery analysis for maritime traffic pattern recognition, Federated Learning across shipping fleets for collaborative model improvement
SIH1515	Ministry of Housing and Urban Affairs	Software	Smart and Effective realtime Management of street parking	Frontend: React.js / React Native (mobile), Leaflet.js / Mapbox for parking spot map, HTML5, CSS3, JavaScript Backend: Python (FastAPI / Django), Node.js, WebSocket for real-time updates Database: PostgreSQL + PostGIS (parking zones, spots), Redis (real-time occupancy cache), TimescaleDB (historical data) Detection: Computer Vision (YOLOv8) on existing CCTV for occupancy detection, OR IoT sensors (ultrasonic/magnetometer per spot), OR crowdsourced user check-in/check-out Payment: Razorpay / PayU integration, UPI, dynamic pricing engine based on demand Features: Spot reservation, navigation to spot (Google Maps SDK), violation detection (overstay/no-payment), enforcement officer mobile app Analytics: Occupancy rate trends, revenue analytics, peak hour analysis, zone-wise heatmaps Notifications: Spot availability alerts, parking expiry reminder Auth: JWT, Firebase Auth Deployment: Docker, Nginx, edge AI for CCTV processing	LiDAR-based 3D parking spot mapping, Deep Reinforcement Learning for dynamic pricing optimization, Digital Twin of urban parking ecosystem, Autonomous vehicle parking integration (V2I communication), Blockchain-based parking payment and violation records
SIH1516	Ministry of Housing and Urban Affairs	Software	Suggest an AI-based solution to enable ease of grievance lodging and tracking for citizens across multiple departments	Frontend: React.js / Next.js (PWA), HTML5, CSS3, JavaScript, multi-step grievance form, status tracker UI Backend: Python (Django / FastAPI), Node.js Database: PostgreSQL (grievances, departments, status), Elasticsearch (grievance search), Redis NLP/AI: Text classification (BERT / IndicBERT) for auto-categorization to department, sentiment analysis for priority scoring, NER (spaCy) for location/entity extraction from complaint text, duplicate detection (sentence-transformers similarity) Chatbot: LangChain + RAG for guided grievance filing, Bhashini API for multilingual support Workflow: Multi-department routing engine, SLA tracking, escalation on SLA breach, inter-department transfer Mobile: React Native / Flutter Notifications: SMS, email, push for status updates Analytics: Department-wise resolution metrics, citizen satisfaction scoring, geographic grievance heatmap Auth: Aadhaar-based optional, JWT, guest filing option Deployment: Docker, Nginx, NIC Cloud	LLM-powered auto-drafting of grievance from voice input, Predictive resolution time estimation using historical patterns, Blockchain-based transparent grievance lifecycle tracking, AI-powered department recommendation with 95%+ accuracy, Citizen sentiment dashboard for policy makers

LEGEND & NOTES

Column	Description
PS No.	Official SIH Problem Statement Number
Organization	Government ministry or organization that submitted the problem
Cat.	Category: Software, Hardware, or Software+Hardware
Problem Statement	Complete problem statement title/description from official SIH documents
Standard Tech Stack	The reliable, proven technology stack recommended for building a competition-winning solution. Covers frontend, backend, database, AI/ML frameworks, specific algorithms, libraries, hardware components, communication protocols, and deployment tools. These are practical choices that work and can be demonstrated in a hackathon.
Advanced / Buzzword Tech	Cutting-edge technologies and approaches that can be mentioned in presentations to impress judges, or partially implemented as stretch goals. These are more research-oriented and may not be fully implementable in a hackathon timeframe, but demonstrate technical depth and vision.

Generated for SIH Smart Automation Theme Research | All problem statements extracted from official SIH documents